Python activity based learning

Case study documentation

Each case study is prepared in the required format:

1.problem statement

2.objectives

3.assumptions

4.algorithum

5.flowchart

6.pseudocode

7.python source code

8.test cases

9.output

10.conclusion

Selected Case study Names

1. Discount Calculation System

2.ATM Transaction Simulation System

3.Advanced Numeric Pattern Generator

4.Student Attendance and Internal Marks Evaluation System

5.Traffic Violation Penalty Calculator

6.Automated Courier Charge Calculation System

7.Student Email and ID Validation System

8.Calendar Generation Using Nested Loops

9.University Scholarship Allocation System

10.Campus Transport Allocation System

Case Study 1: Retail Discount Calculation System

1. Problem Statement

Develop a Python program to calculate the discount and final purchase amount based on the purchase amount. If the purchase amount is greater

than ₹5,000, a 20% discount is applied. If the purchase amount is greater than ₹2,000 but less than or equal to ₹5,000, a 10% discount is applied. Otherwise, no discount is given. Finally, display the purchase amount, discount, and final amount.

2. Objective

1. To calculate discounts based on the purchase amount.
2. To understand the use of if, elif, and else statements in Python.
3. To calculate the final amount after deducting the discount.
4. To display the purchase amount, discount, and final amount.

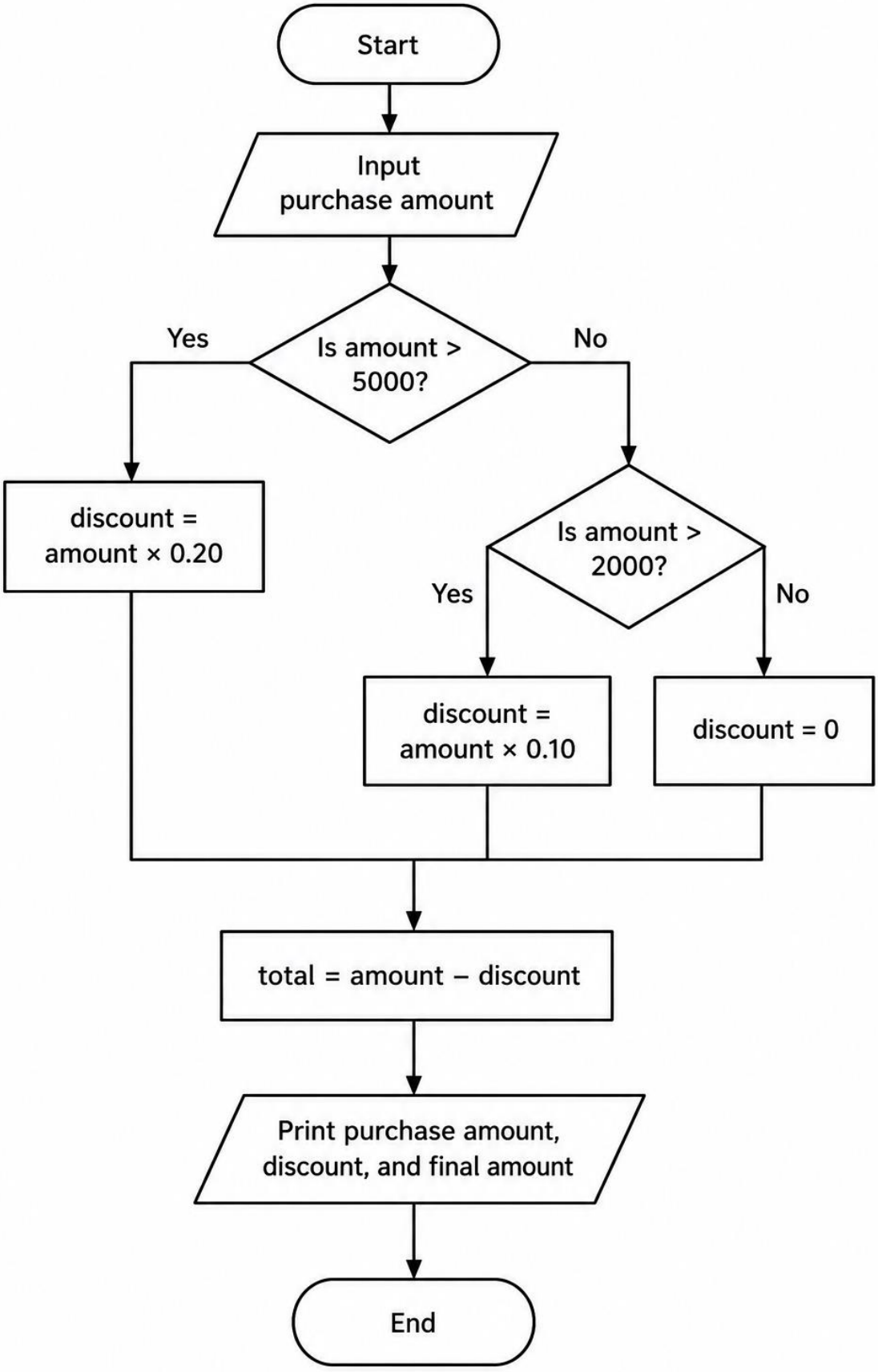
3. Assumptions

1. The purchase amount is entered by the user.
2. The purchase amount is a numeric value.
3. A 20% discount is given when the purchase amount exceeds ₹5,000.
4. A 10% discount is given when the purchase amount exceeds ₹2,000 and is at most ₹5,000.
5. No discount is given when the purchase amount is ₹2,000 or less.
6. The purchase amount is assumed to be non-negative.

4. Algorithm

1. Start.
2. Read the purchase amount.
3. Check whether the purchase amount is greater than 5,000.
4. If true, calculate the discount as 20% of the purchase amount.
5. Otherwise, check whether the purchase amount is greater than 2,000.
6. If true, calculate the discount as 10% of the purchase amount.
7. Otherwise, set the discount to zero.
8. Calculate the final amount by subtracting the discount from the purchase amount.
9. Display the purchase amount, discount, and final amount.
10. Stop.

5.Flowchart


Start
Input purchase amount
Is amount > 5000?
Yes
No
discount = amount × 0.20
Is amount > 2000?
Yes
No
discount = amount × 0.10
discount = 0
total = amount − discount
Print purchase amount, discount, and final amount
End

6. Pseudocode

```
START

INPUT amount

IF amount > 5000 THEN
   discount = amount * 0.20

ELSE IF amount > 2000 THEN
   discount = amount * 0.10

ELSE
   discount = 0

END IF

total = amount - discount

PRINT "Purchase Amount =", amount
PRINT "Discount =", discount
PRINT "Final Amount =", total

STOP
```

7.Python Source Code

```
amount = float(input("Enter purchase amount: "))

if amount > 5000:
   discount = amount * 0.20
elif amount > 2000:
   discount = amount * 0.10
else:
```

```
    discount = 0

total = amount - discount

print("Purchase Amount =", amount)
print("Discount =", discount)
print("Final Amount =", total)
```

8.Test cases

Test case 1

Enter purchase amount: 6000

Purchase Amount = 6000.0

Discount = 1200.0

Final Amount = 4800.0

Test case 2

Enter purchase amount: 3000

Purchase Amount = 3000.0

Discount = 300.0

Final Amount = 2700.0

Test case 3

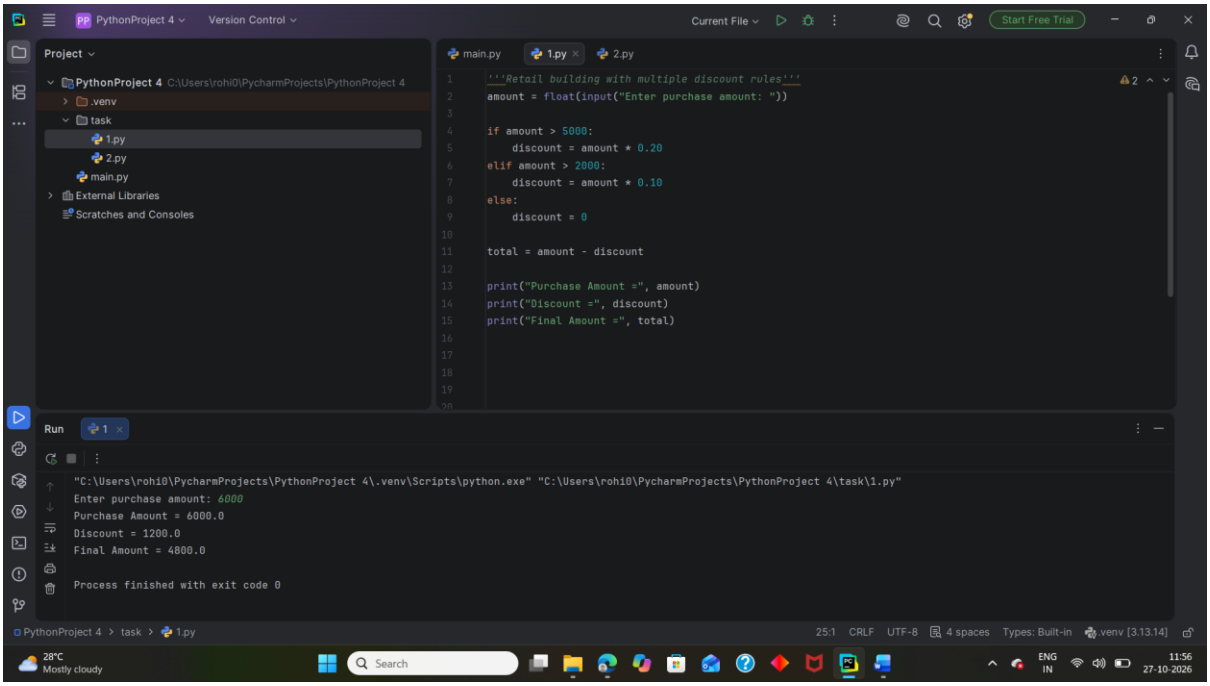
Enter purchase amount: 1500

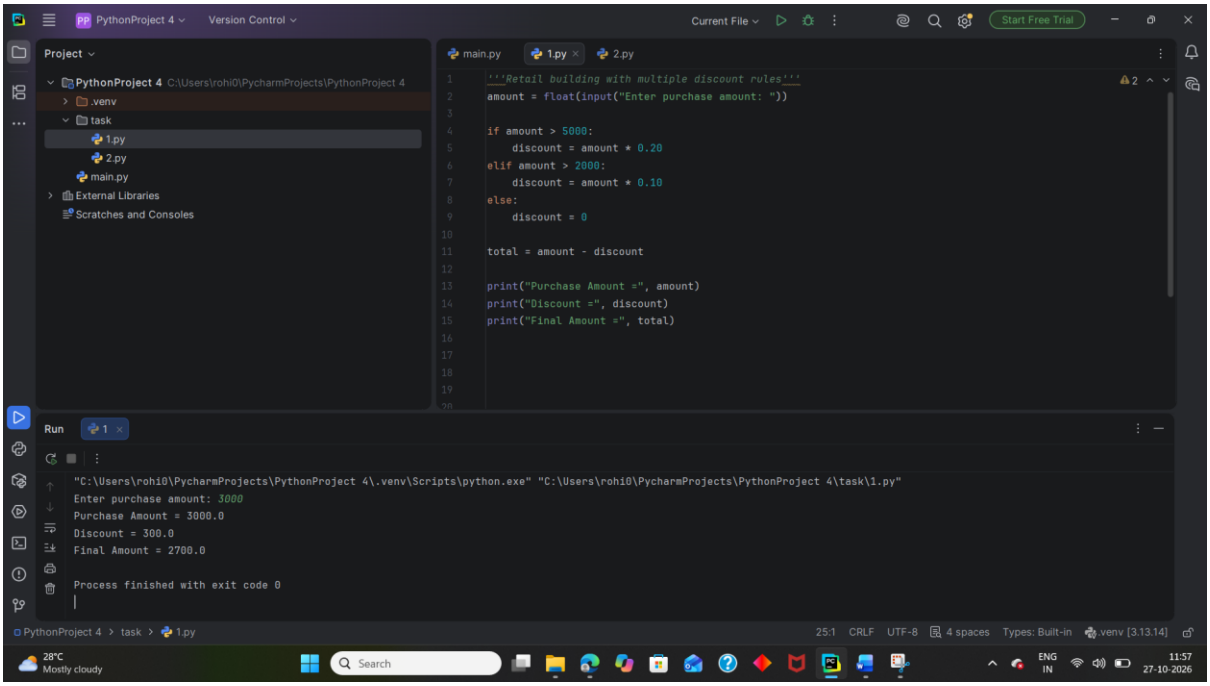
Purchase Amount = 1500.0

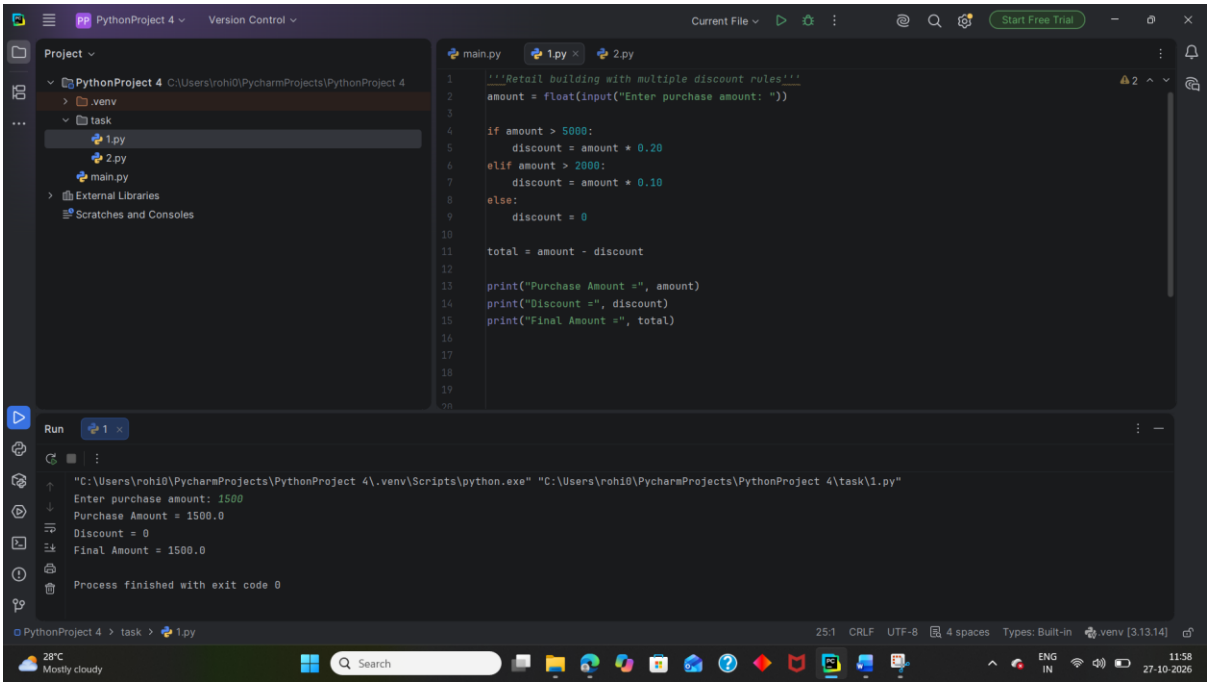
Discount = 0

Final Amount = 1500.0

9.output


PythonProject 4
Version Control
Current File
Start Free Trial
Project
PythonProject 4 C:\Users\rohi0\PycharmProjects\PythonProject 4
.venv
task
1.py
2.py
main.py
External Libraries
Scratches and Consoles
main.py
1.py
2.py
'''Retail building with multiple discount rules'''
amount = float(input("Enter purchase amount: "))
if amount > 5000:
discount = amount * 0.20
elif amount > 2000:
discount = amount * 0.10
else:
discount = 0
total = amount - discount
print("Purchase Amount =", amount)
print("Discount =", discount)
print("Final Amount =", total)
Run
"C:\Users\rohi0\PycharmProjects\PythonProject 4\.venv\Scripts\python.exe" "C:\Users\rohi0\PycharmProjects\PythonProject 4\task\1.py"
Enter purchase amount: 6000
Purchase Amount = 6000.0
Discount = 1200.0
Final Amount = 4800.0
Process finished with exit code 0
PythonProject 4 > task > 1.py
25:1 CRLF UTF-8 4 spaces Types: Built-in venv [3.13.14]
28°C
Mostly cloudy
Search
ENG
IN
11:56
27-10-2026


PythonProject 4
Version Control
Current File
Start Free Trial
Project
PythonProject 4 C:\Users\rohi0\PycharmProjects\PythonProject 4
.venv
task
1.py
2.py
main.py
External Libraries
Scratches and Consoles
main.py
1.py
2.py
'''Retail building with multiple discount rules'''
amount = float(input("Enter purchase amount: "))
if amount > 5000:
discount = amount * 0.20
elif amount > 2000:
discount = amount * 0.10
else:
discount = 0
total = amount - discount
print("Purchase Amount =", amount)
print("Discount =", discount)
print("Final Amount =", total)
Run
"C:\Users\rohi0\PycharmProjects\PythonProject 4\.venv\Scripts\python.exe" "C:\Users\rohi0\PycharmProjects\PythonProject 4\task\1.py"
Enter purchase amount: 3000
Purchase Amount = 3000.0
Discount = 300.0
Final Amount = 2700.0
Process finished with exit code 0
PythonProject 4 > task > 1.py
25:1 CRLF UTF-8 4 spaces Types: Built-in venv [3.13.14]
28°C
Mostly cloudy
Search
ENG
IN
11:57
27-10-2026



10.Conclusion: This program calculates the discount and final purchase amount using conditional statements and arithmetic operations in Python.

Case Study 2: ATM Transaction Simulation System

1. Problem Statement

Develop a Python program to simulate an ATM withdrawal transaction. The program should accept the account balance and withdrawal amount from the user. If the withdrawal amount is less than or equal to zero, display an invalid withdrawal message. If the withdrawal amount exceeds the account balance, display an insufficient balance message. Otherwise, deduct the withdrawal amount from the balance and display the withdrawal success message and remaining balance.

2. Objective

1. To simulate an ATM withdrawal transaction using Python.
2. To validate the withdrawal amount.
3. To check whether the account has sufficient balance.
4. To calculate and display the remaining balance.

5. To understand the use of if, elif, and else statements.

3. Assumptions

1. The user enters the account balance and withdrawal amount.
2. Both values are numeric.
3. The account balance is non-negative.
4. The withdrawal amount must be greater than zero.
5. The withdrawal amount must not exceed the available balance.
6. The transaction is successful only when all conditions are satisfied.

4. Algorithm

1. Start.
2. Input the account balance.
3. Input the withdrawal amount.
4. Check whether the withdrawal amount is less than or equal to zero.

If true, display Invalid Withdrawal Amount.

5. Otherwise, check whether the withdrawal amount is greater than the account balance.

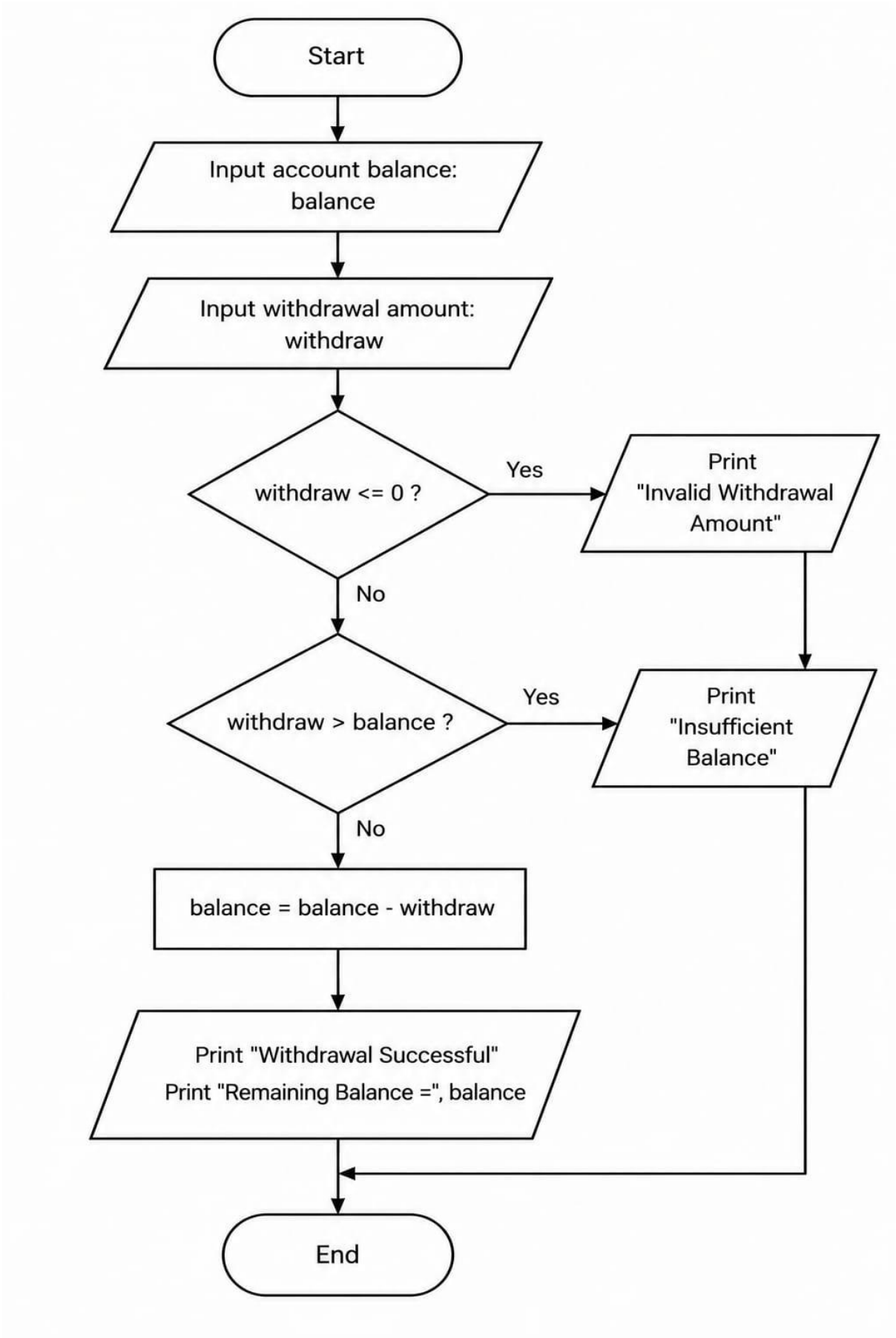
If true, display Insufficient Balance.

6. Otherwise, subtract the withdrawal amount from the account balance.

Display Withdrawal Successful.

7. Display the remaining balance.
8. Stop.

5.Flowchart


Start
Input account balance: balance
Input withdrawal amount: withdraw
withdraw <= 0 ?
Yes
Print "Invalid Withdrawal Amount"
No
withdraw > balance ?
Yes
Print "Insufficient Balance"
No
balance = balance - withdraw
Print "Withdrawal Successful"
Print "Remaining Balance =", balance
End

6. Pseudocode

```
START

INPUT balance
INPUT withdraw

IF withdraw <= 0 THEN
   PRINT "Invalid Withdrawal Amount"

ELSE IF withdraw > balance THEN
   PRINT "Insufficient Balance"

ELSE
   balance = balance - withdraw
   PRINT "Withdrawal Successful"
   PRINT "Remaining Balance =", balance

END IF

STOP
```

7.Python Source Code

```
balance = float(input("Enter account balance: "))
withdraw = float(input("Enter withdrawal amount: "))

if withdraw <= 0:
    print("Invalid Withdrawal Amount")
elif withdraw > balance:
    print("Insufficient Balance")
else:
    balance = balance - withdraw
    print("Withdrawal Successful")
    print("Remaining Balance =", balance)
```

8.Test cases

Test case 1

Enter account balance: 5000.0

Enter withdrawal amount: 2000.0

Withdrawal Successful

Remaining Balance = 3000.0

Test Case 2

Enter account balance: 5000.0

Enter withdrawal amount: 300.0

Withdrawal Successful

Remaining Balance = 4700.0

Test Case 3

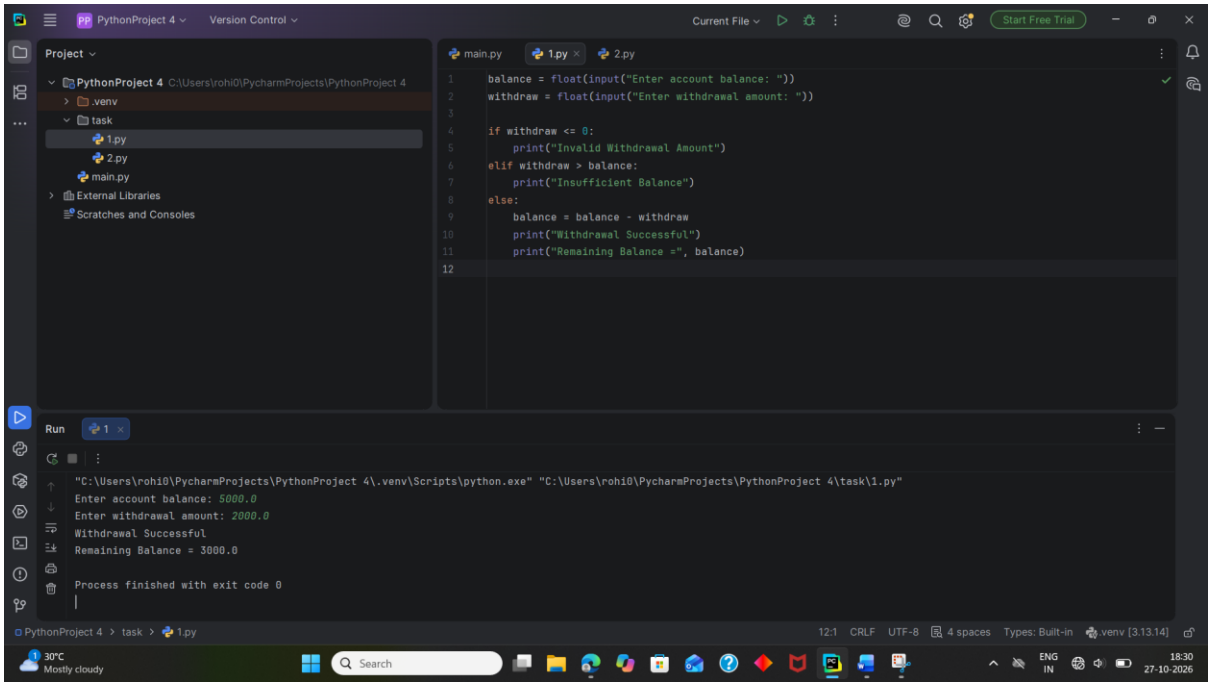
Enter account balance: 4000.0

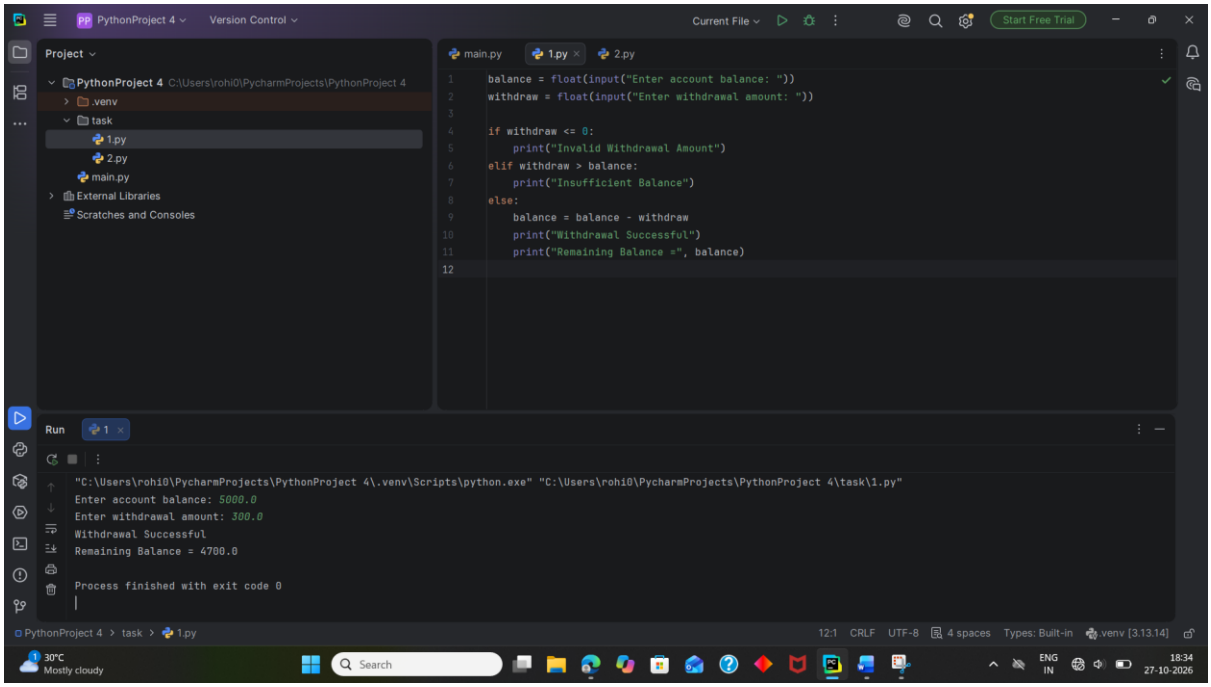
Enter withdrawal amount: 800.0

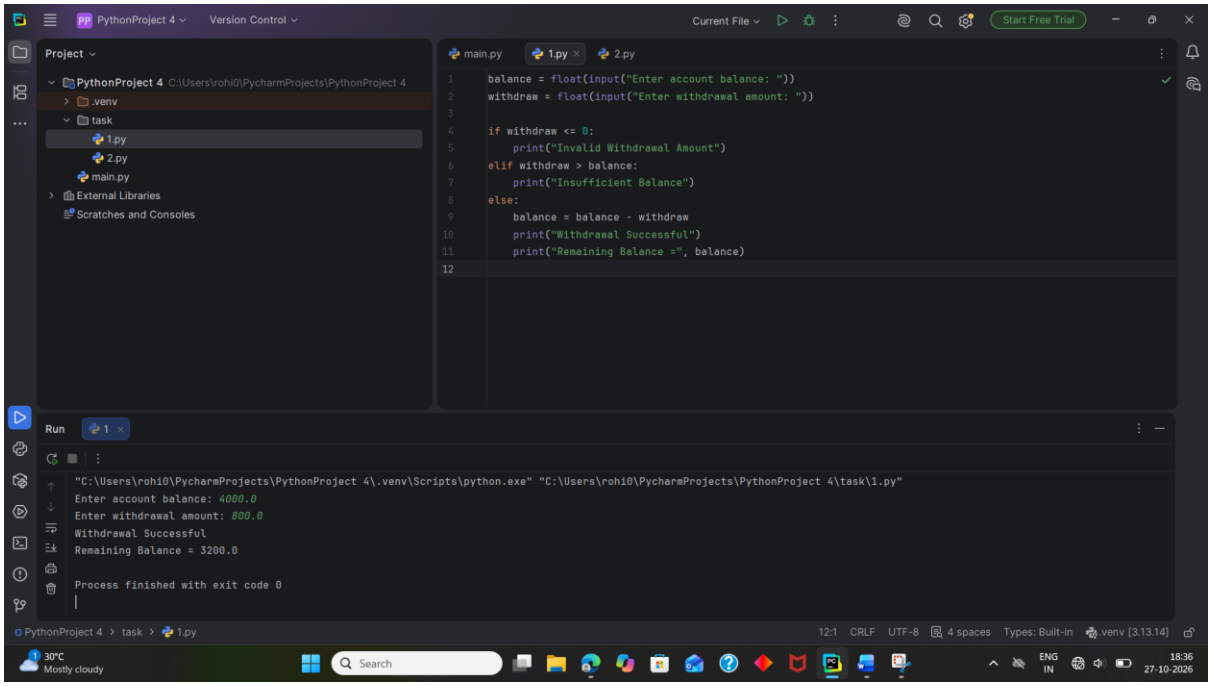
Withdrawal Successful

Remaining Balance = 3200.0

9.Output


PythonProject 4
Version Control
Current File
Start Free Trial
Project
PythonProject 4 C:\Users\rohi0\PycharmProjects\PythonProject 4
.venv
task
1.py
2.py
main.py
External Libraries
Scratches and Consoles
main.py
1.py
2.py
balance = float(input("Enter account balance: "))
withdraw = float(input("Enter withdrawal amount: "))
if withdraw <= 0:
print("Invalid Withdrawal Amount")
elif withdraw > balance:
print("Insufficient Balance")
else:
balance = balance - withdraw
print("Withdrawal Successful")
print("Remaining Balance =", balance)
Run
1
"C:\Users\rohi0\PycharmProjects\PythonProject 4\.venv\Scripts\python.exe" "C:\Users\rohi0\PycharmProjects\PythonProject 4\task\1.py"
Enter account balance: 5000.0
Enter withdrawal amount: 2000.0
Withdrawal Successful
Remaining Balance = 3000.0
Process finished with exit code 0
PythonProject 4 > task > 1.py
12:1 CRLF UTF-8 4 spaces Types: Built-in .venv (3.13.14)
30°C
Mostly cloudy
Search
ENG
IN
18:30
27-10-2026


PythonProject 4
Version Control
Current File
Start Free Trial
Project
PythonProject 4 C:\Users\rohi0\PycharmProjects\PythonProject 4
.venv
task
1.py
2.py
main.py
External Libraries
Scratches and Consoles
main.py
1.py
2.py
balance = float(input("Enter account balance: "))
withdraw = float(input("Enter withdrawal amount: "))
if withdraw <= 0:
print("Invalid Withdrawal Amount")
elif withdraw > balance:
print("Insufficient Balance")
else:
balance = balance - withdraw
print("Withdrawal Successful")
print("Remaining Balance =", balance)
Run
1
"C:\Users\rohi0\PycharmProjects\PythonProject 4\.venv\Scripts\python.exe" "C:\Users\rohi0\PycharmProjects\PythonProject 4\task\1.py"
Enter account balance: 5000.0
Enter withdrawal amount: 300.0
Withdrawal Successful
Remaining Balance = 4700.0
Process finished with exit code 0
PythonProject 4 > task > 1.py
12:1 CRLF UTF-8 4 spaces Types: Built-in .venv (3.13.14)
30°C
Mostly cloudy
Search
ENG
IN
18:34
27-10-2026



10. Conclusion

The ATM Transaction Simulation System demonstrates how Python conditional statements can validate withdrawal amounts, check account balances, and perform transactions. It calculates and displays the remaining balance when a withdrawal is successful.

Case Study 3: Advanced Numeric Pattern Generator

1. Problem Statement

Develop a Python program to generate a numeric pattern using nested for loops. The program should accept an integer n from the user and print numbers from 1 to the current row number. The pattern should contain n rows.

2. Objective

1. To generate a numeric pattern using Python.
2. To understand the use of nested for loops.
3. To print numbers in a structured pattern.
4. To improve logical thinking and programming skills.

3. Assumptions

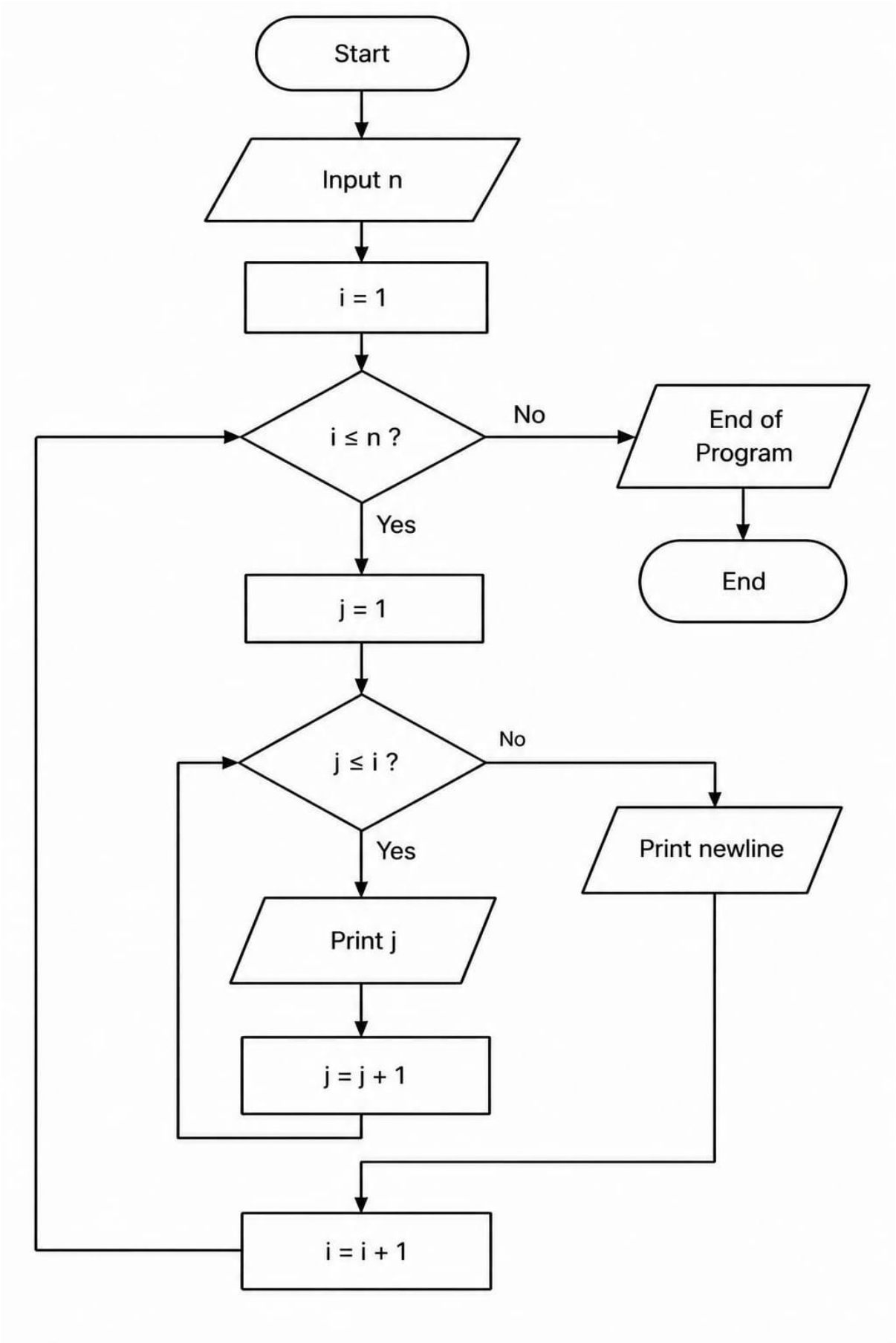
1. The user enters an integer value n.
2. The value of n is a positive integer.
3. Each row starts with the number 1.

4. Each row contains numbers up to its row number.
5. The pattern contains n rows.

4. Algorithm

1. Start.
2. Read the number of rows n.
3. Initialize the outer loop variable i from 1 to n.
4. For each row, initialize the inner loop variable j from 1 to i.
5. Print j on the same line with a space.
6. After the inner loop finishes, move to the next line.
7. Repeat until all rows are printed.
8. Stop.

5.Flowchart


Start
Input n
i = 1
i ≤ n ?
No
End of Program
End
Yes
j = 1
j ≤ i ?
No
Print newline
Yes
Print j
j = j + 1
i = i + 1

6. Pseudocode

```
START

INPUT n

FOR i = 1 TO n
   FOR j = 1 TO i
      PRINT j WITHOUT NEWLINE
   END FOR

   PRINT NEWLINE
END FOR

STOP
```

7. Python Source Code

```
# Advanced Numeric Pattern Generator

n = int(input("Enter the number of rows: "))

for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

8.Test Cases

Test case 1

4

1

1 2

1 2 3

1 2 3 4

Test case 2

7

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

1 2 3 4 5 6

1 2 3 4 5 6 7

Test case 3

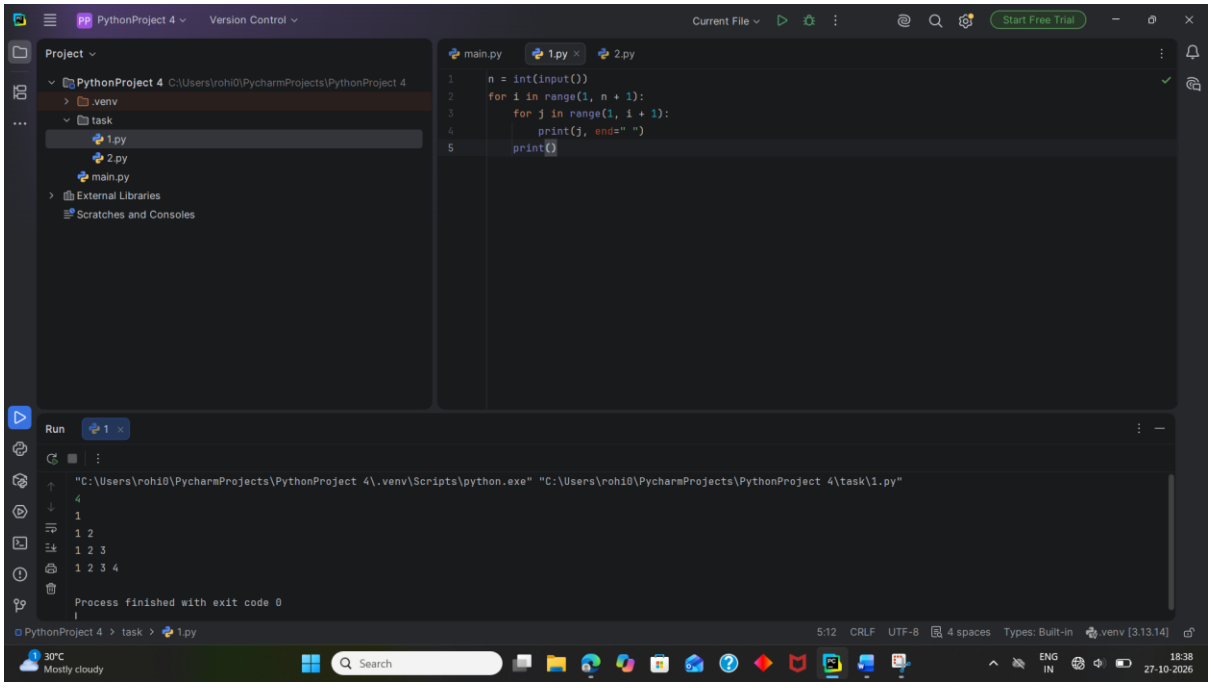
3

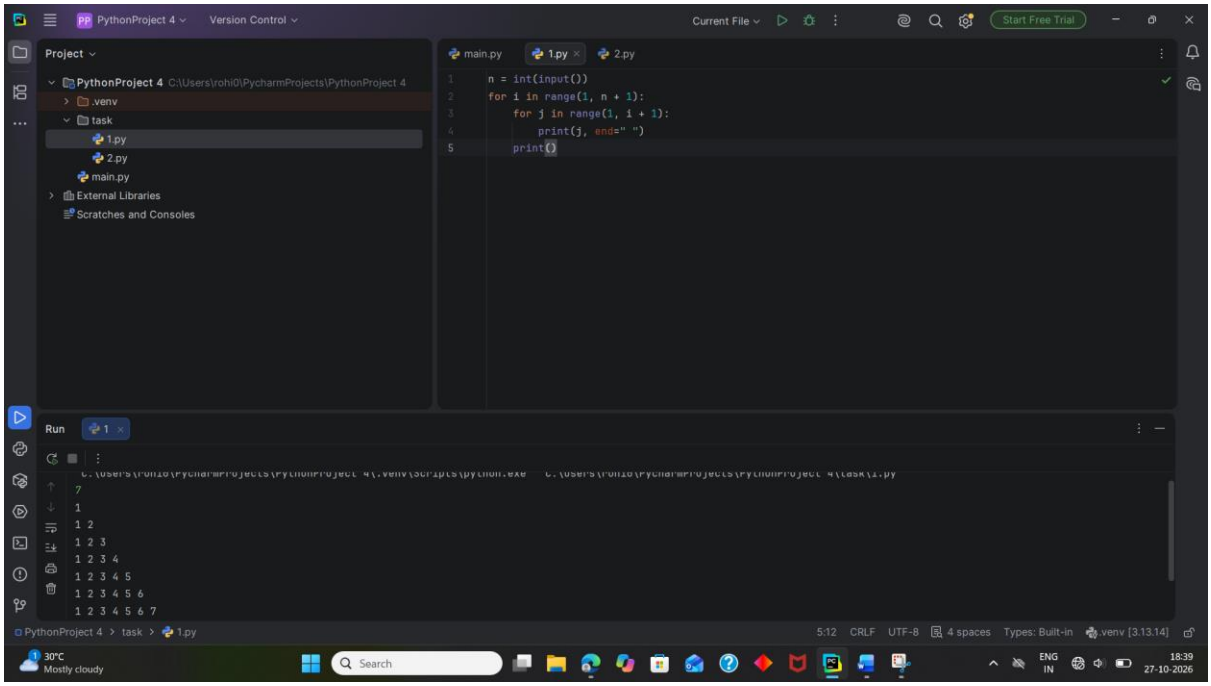
1

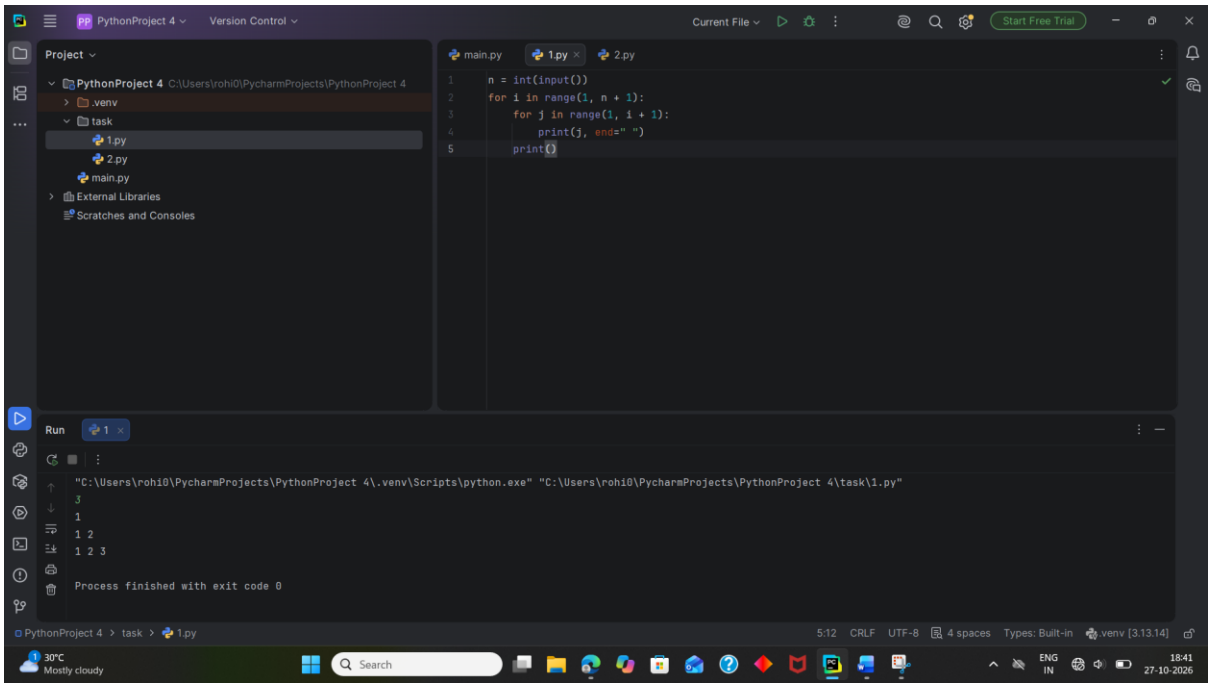
1 2

1 2 3

9.output







10. Conclusion

The Advanced Numeric Pattern Generator successfully prints a numeric pattern using nested for loops. The outer loop controls the rows, while the inner loop prints numbers in each row. This program helps develop an understanding of loops and pattern generation in Python.

Case Study 4: Student Attendance and Internal Marks Evaluation System

1. Problem Statement

Develop a Python program to determine students' examination eligibility based on their attendance percentage and internal marks. If attendance is at least 75% and internal marks are at least 40, the student is eligible for the examination. If attendance is sufficient but marks are below 40, display "Failed in Internal Marks". If attendance is below 75%, display "Insufficient Attendance".

2. Objective

1. To check student examination eligibility.
2. To evaluate attendance percentage and internal marks.
3. To understand nested if-else statements in Python.
4. To use a for loop to process multiple students.
5. To display the appropriate result for each student.

3. Assumptions

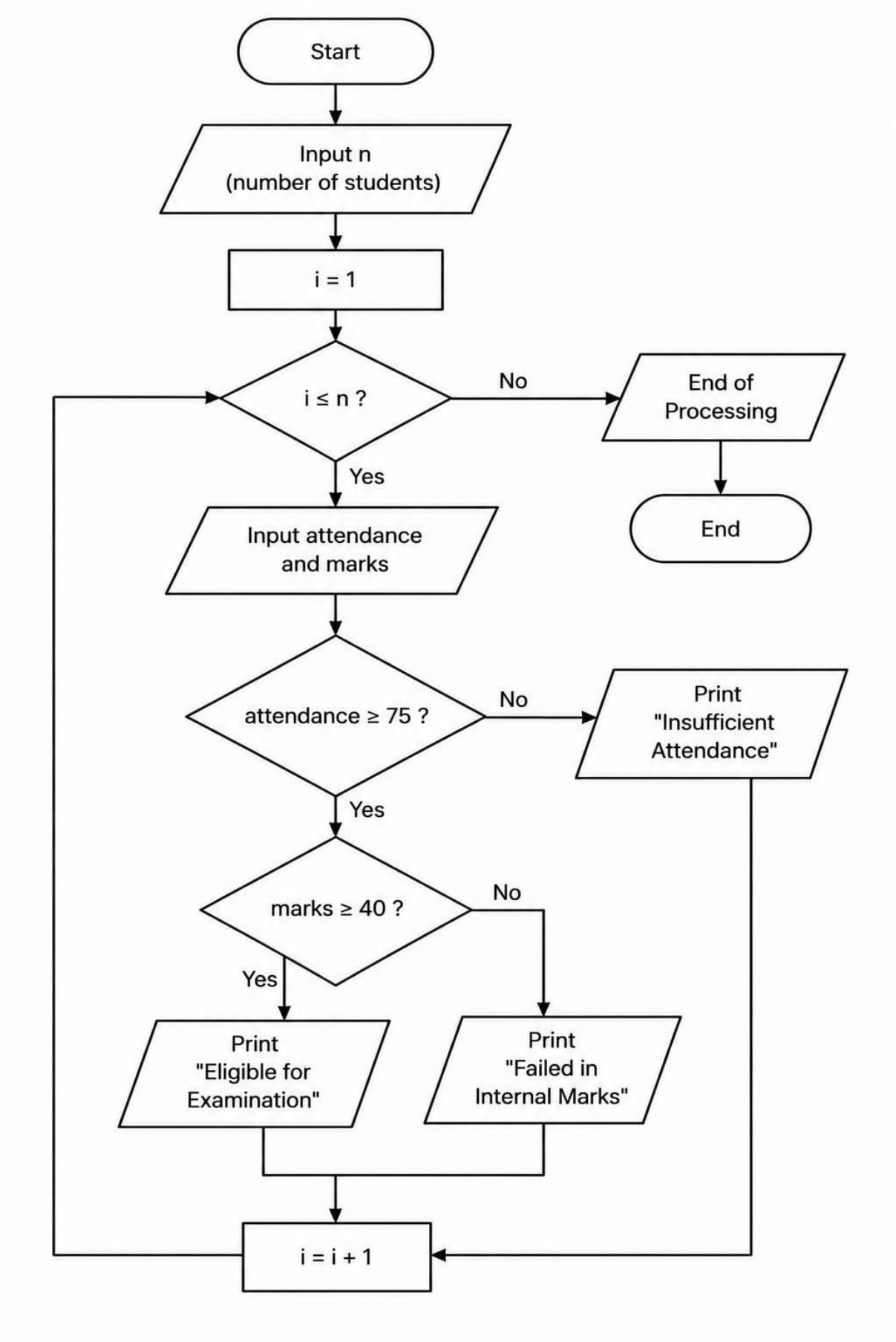
1. The number of students is stored in the variable n.
2. Attendance is entered as a percentage.
3. Internal marks are entered as integers.
4. Minimum required attendance is 75%.
5. Minimum required internal marks are 40.
6. The program evaluates each student separately.

4. Algorithm

1. Start.
2. Read the number of students, n.
3. Repeat the following steps for each student from 1 to n:
    - Read the attendance percentage.
    - Read the internal marks.
    - Check whether attendance is greater than or equal to 75.
    - If true, check whether marks are greater than or equal to 40.
    - If both conditions are true, display "Eligible for Examination".

    - If attendance is sufficient but marks are below 40, display "Failed in Internal Marks".
    - If attendance is below 75%, display "Insufficient Attendance".
4. Repeat until all students have been evaluated.
5. Stop.

5.flowchart


Start
Input n
(number of students)
i = 1
i ≤ n ?
No
End of
Processing
End
Yes
Input attendance
and marks
attendance ≥ 75 ?
No
Print
"Insufficient
Attendance"
Yes
marks ≥ 40 ?
No
Yes
Print
"Eligible for
Examination"
Print
"Failed in
Internal Marks"
i = i + 1

6. Pseudocode

```
START

INPUT n

FOR i = 1 TO n
   INPUT attendance
   INPUT marks

   IF attendance >= 75 THEN
      IF marks >= 40 THEN
         PRINT "Eligible for Examination"
      ELSE
         PRINT "Failed in Internal Marks"
      END IF
   ELSE
      PRINT "Insufficient Attendance"
   END IF

END FOR

STOP
```

7.Python Source Code

```
n=int(input())
for i in range(1, n + 1):
    attendance = int(input())
    marks = int(input())

    if attendance >= 75:
        if marks >= 40:
            print("Eligible for Examination")
        else:
```

```
            print("Failed in Internal Marks")
    else:
        print("Insufficient Attendance")
```

8.Test cases

Test case 1

85

45

30

Insufficient Attendance

Test case 2

75

80

90

Eligible for Examination

Test case 3

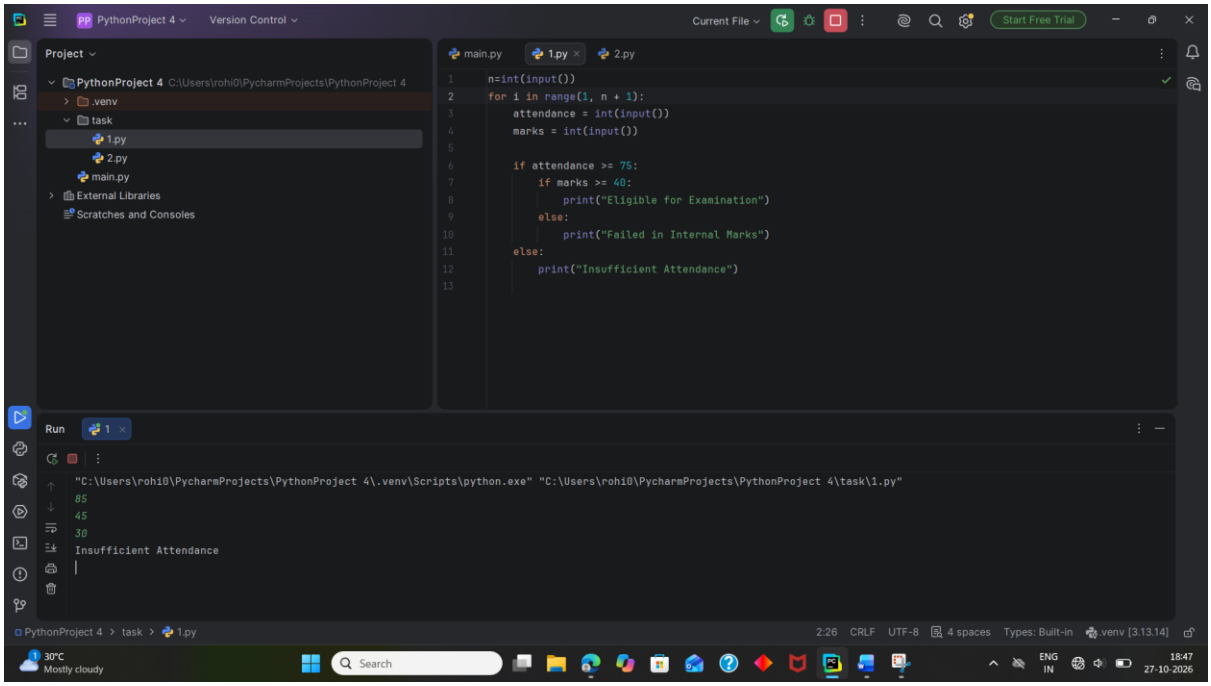
50

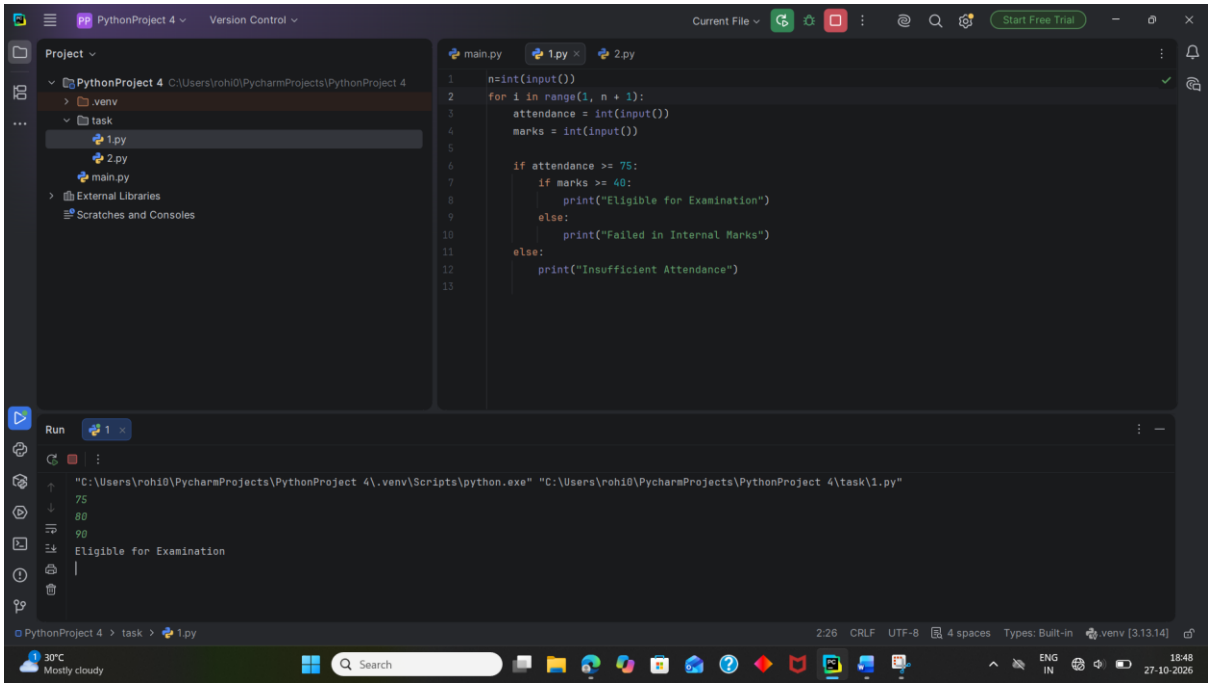
40

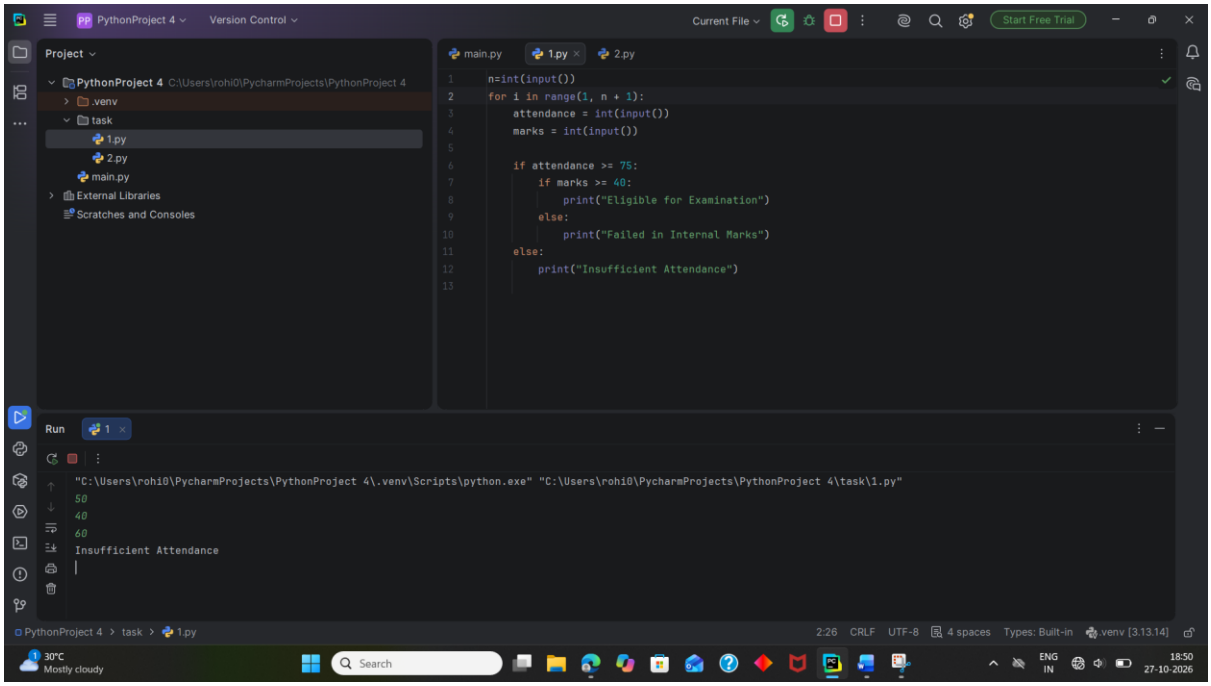
60

Insufficient Attendance

9.Output


PythonProject 4
Version Control
Current File
Start Free Trial
Project
PythonProject 4 C:\Users\rohi0\PycharmProjects\PythonProject 4
.venv
task
1.py
2.py
main.py
External Libraries
Scratches and Consoles
main.py
1.py
2.py
n=int(input())
for i in range(1, n + 1):
attendance = int(input())
marks = int(input())
if attendance >= 75:
if marks >= 40:
print("Eligible for Examination")
else:
print("Failed in Internal Marks")
else:
print("Insufficient Attendance")
Run
1
"C:\Users\rohi0\PycharmProjects\PythonProject 4\.venv\Scripts\python.exe" "C:\Users\rohi0\PycharmProjects\PythonProject 4\task\1.py"
85
45
30
Insufficient Attendance
PythonProject 4 > task > 1.py
2:26 CRLF UTF-8 4 spaces Types: Built-in .venv (3.13.14)
30°C
Mostly cloudy
Search
ENG
IN
18:47
27-10-2026


PythonProject 4
Version Control
Current File
Start Free Trial
Project
PythonProject 4 C:\Users\rohi0\PycharmProjects\PythonProject 4
.venv
task
1.py
2.py
main.py
External Libraries
Scratches and Consoles
main.py
1.py
2.py
n=int(input())
for i in range(1, n + 1):
attendance = int(input())
marks = int(input())
if attendance >= 75:
if marks >= 40:
print("Eligible for Examination")
else:
print("Failed in Internal Marks")
else:
print("Insufficient Attendance")
Run
1
"C:\Users\rohi0\PycharmProjects\PythonProject 4\.venv\Scripts\python.exe" "C:\Users\rohi0\PycharmProjects\PythonProject 4\task\1.py"
75
80
90
Eligible for Examination
PythonProject 4 > task > 1.py
2:26 CRLF UTF-8 4 spaces Types: Built-in .venv (3.13.14)
30°C
Mostly cloudy
Search
ENG
IN
18:48
27-10-2026



10. Conclusion

The Student Attendance and Internal Marks Evaluation System determines examination eligibility based on attendance and internal marks. It uses a for loop and nested if-else statements to evaluate multiple students and display the appropriate result.

Case Study 5: Traffic Violation Penalty Calculator

1. Problem Statement

Develop a Python program to calculate the traffic fine based on the vehicle's speed. If the speed is less than or equal to 60 km/h, no fine is charged. If the speed is greater than 60 km/h but less than or equal to 80 km/h, a fine of ₹500 is charged. If the speed exceeds 80 km/h, a fine of ₹1,000 is charged. Finally, display the traffic fine.

2. Objective

1. To calculate traffic fines based on vehicle speed.
2. To understand the use of if, elif, and else statements.
3. To apply conditional logic in a real-world situation.
4. To display the appropriate traffic fine.

3. Assumptions

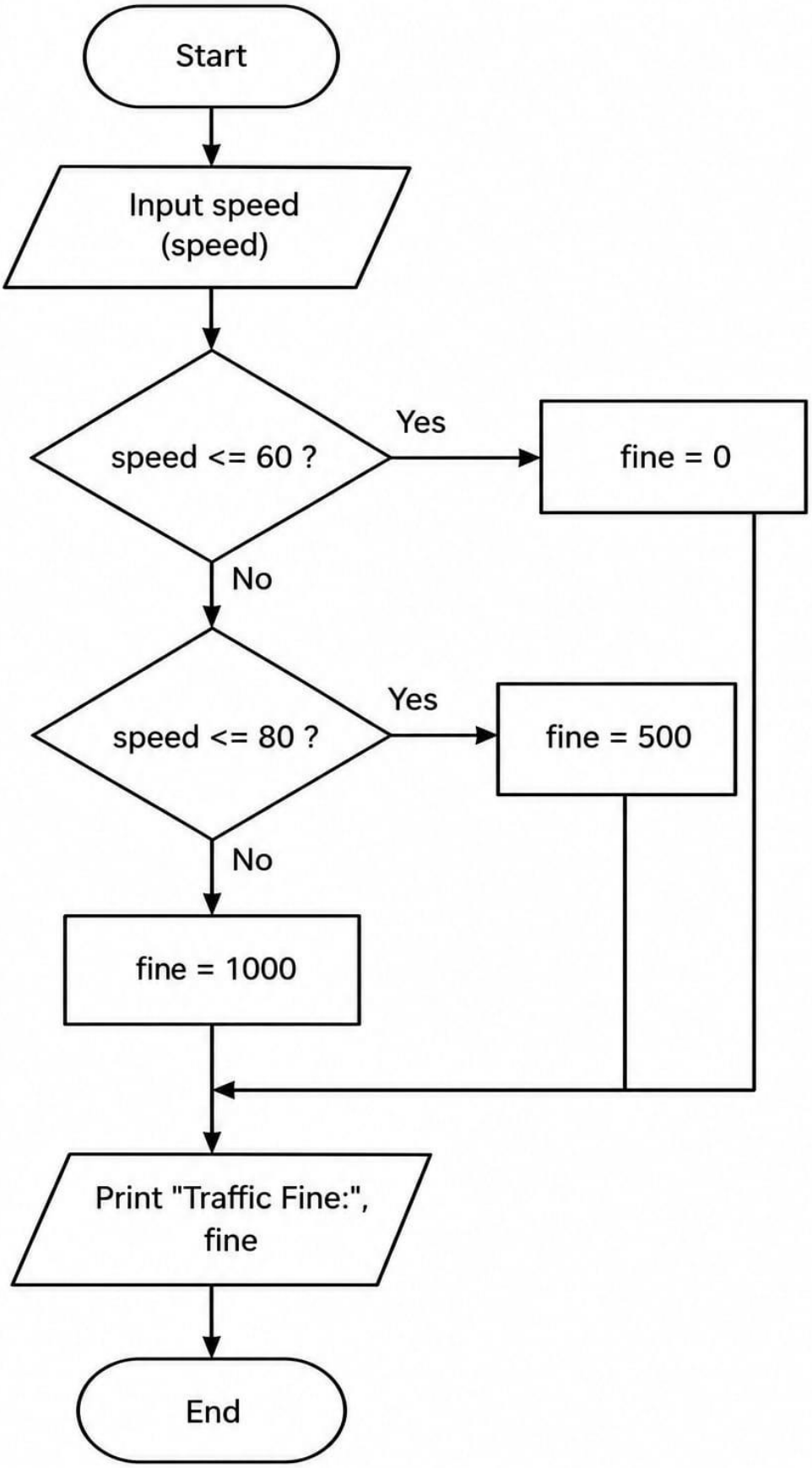
1. The vehicle speed is entered by the user as an integer.
2. The speed is measured in kilometres per hour (km/h).

3. No fine is charged for speeds up to 60 km/h.
4. A fine of ₹500 is charged for speeds above 60 km/h and up to 80 km/h.
5. A fine of ₹1,000 is charged for speeds above 80 km/h.
6. The entered speed is non-negative.

4. Algorithm

1. Start.
2. Read the vehicle speed.
3. Check whether the speed is less than or equal to 60.
4. If true, set the fine to 0.
5. Otherwise, check whether the speed is less than or equal to 80.
6. If true, set the fine to ₹500.
7. Otherwise, set the fine to ₹1,000.
8. Display the traffic fine.
9. Stop.

5.flowchart


Start
Input speed (speed)
speed <= 60 ?
Yes
fine = 0
No
speed <= 80 ?
Yes
fine = 500
No
fine = 1000
Print "Traffic Fine:", fine
End

6. Pseudocode

START

INPUT speed

IF speed <= 60 THEN

fine = 0

ELSE IF speed <= 80 THEN

fine = 500

ELSE

fine = 1000

END IF

PRINT "Traffic Fine:", fine

STOP

7.Python Source Code

```
speed = int(input())

if speed <= 60:
    fine = 0
elif speed <= 80:
    fine = 500
else:
    fine = 1000

print("Traffic Fine:", fine)
```

8.Test Cases

Test case 1

85

Traffic Fine: 1000

Test case 2

80

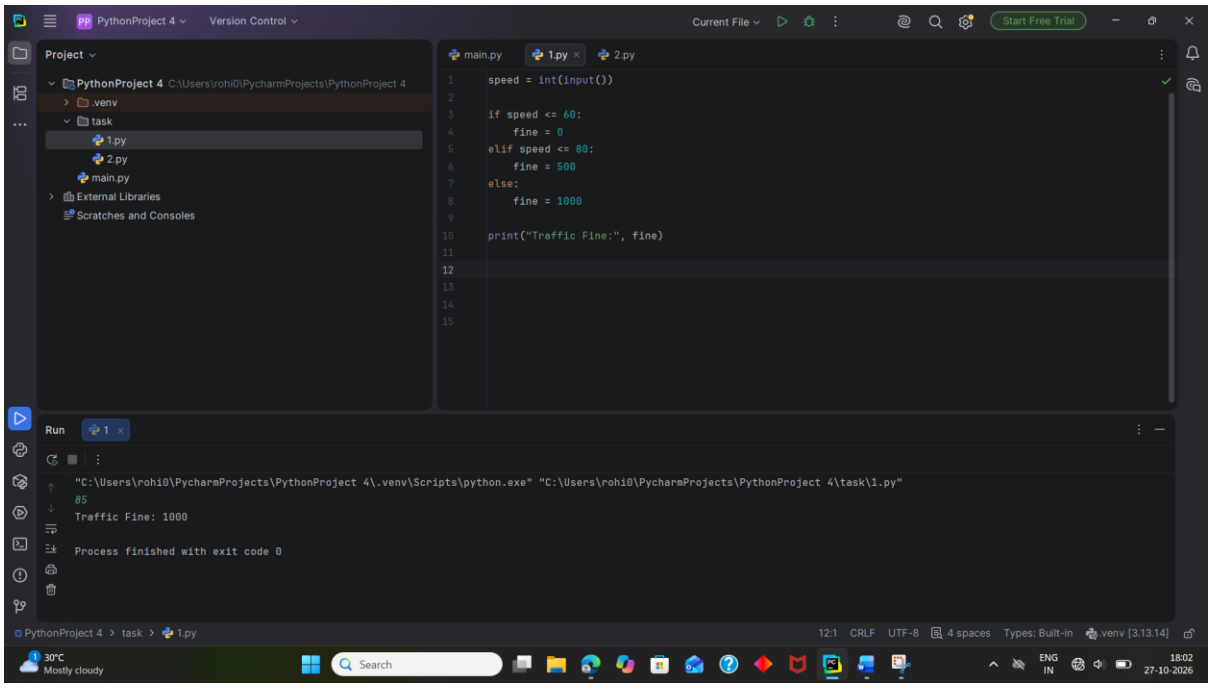
Traffic Fine: 500

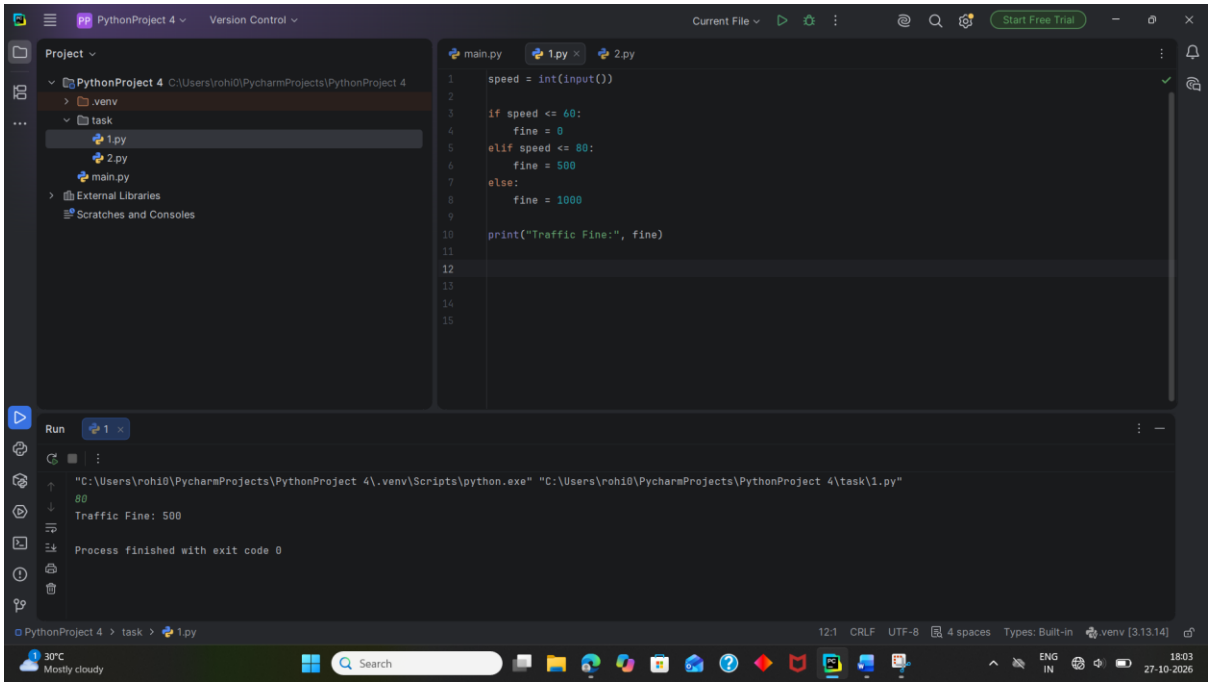
Test case 3

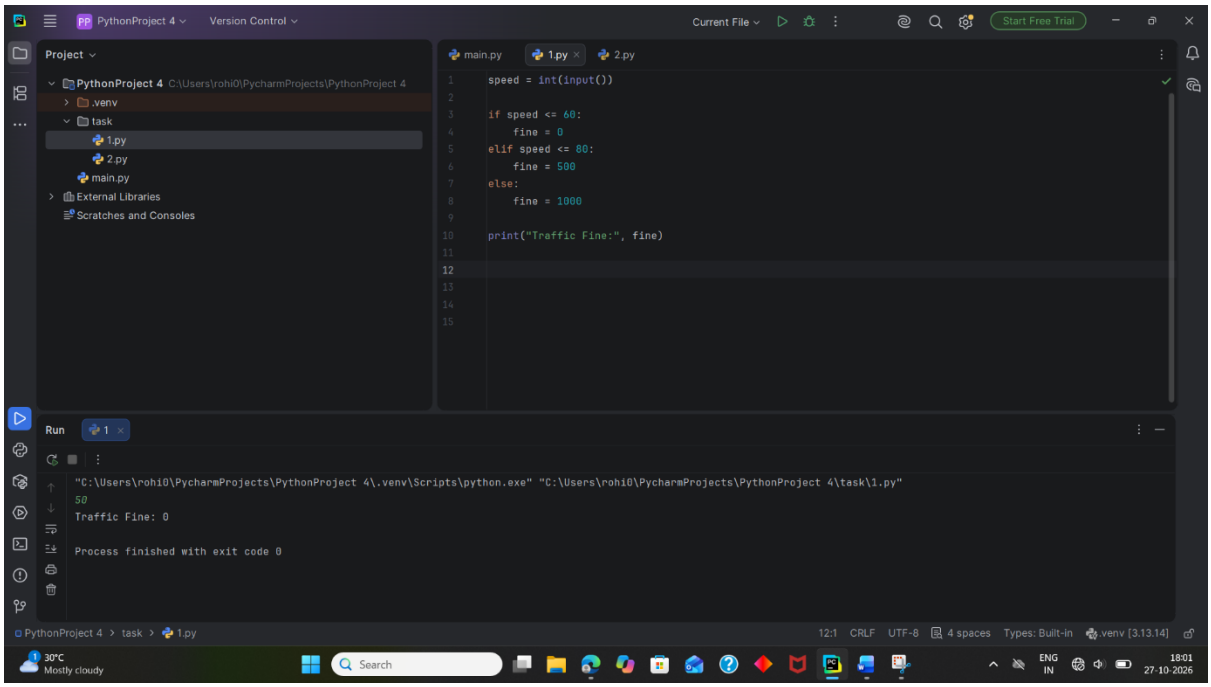
50

Traffic Fine: 0

9.Output







10. Conclusion

The Traffic Violation Penalty Calculator calculates the traffic fine based on the vehicle's speed using if, elif, and else statements. It demonstrates how conditional statements can be used to solve real-world problems in Python.

Case Study 6: Automated Courier Charge Calculation System

1. Problem Statement

Develop a Python program to calculate courier charges based on the weight of a parcel. If the parcel weight is less than or equal to 1 kg, the charge is ₹50. If the weight is greater than 1 kg but less than or equal to 5 kg, the charge is ₹100. If the weight exceeds 5 kg, an additional ₹20 is charged for every kilogram above 5 kg, along with a base charge of ₹100. Finally, display the total courier charge.

2. Objective

1. To calculate courier charges based on parcel weight.
2. To understand the use of if, elif, and else statements.
3. To perform arithmetic calculations using Python.
4. To apply conditional statements to a real-world problem.

3. Assumptions

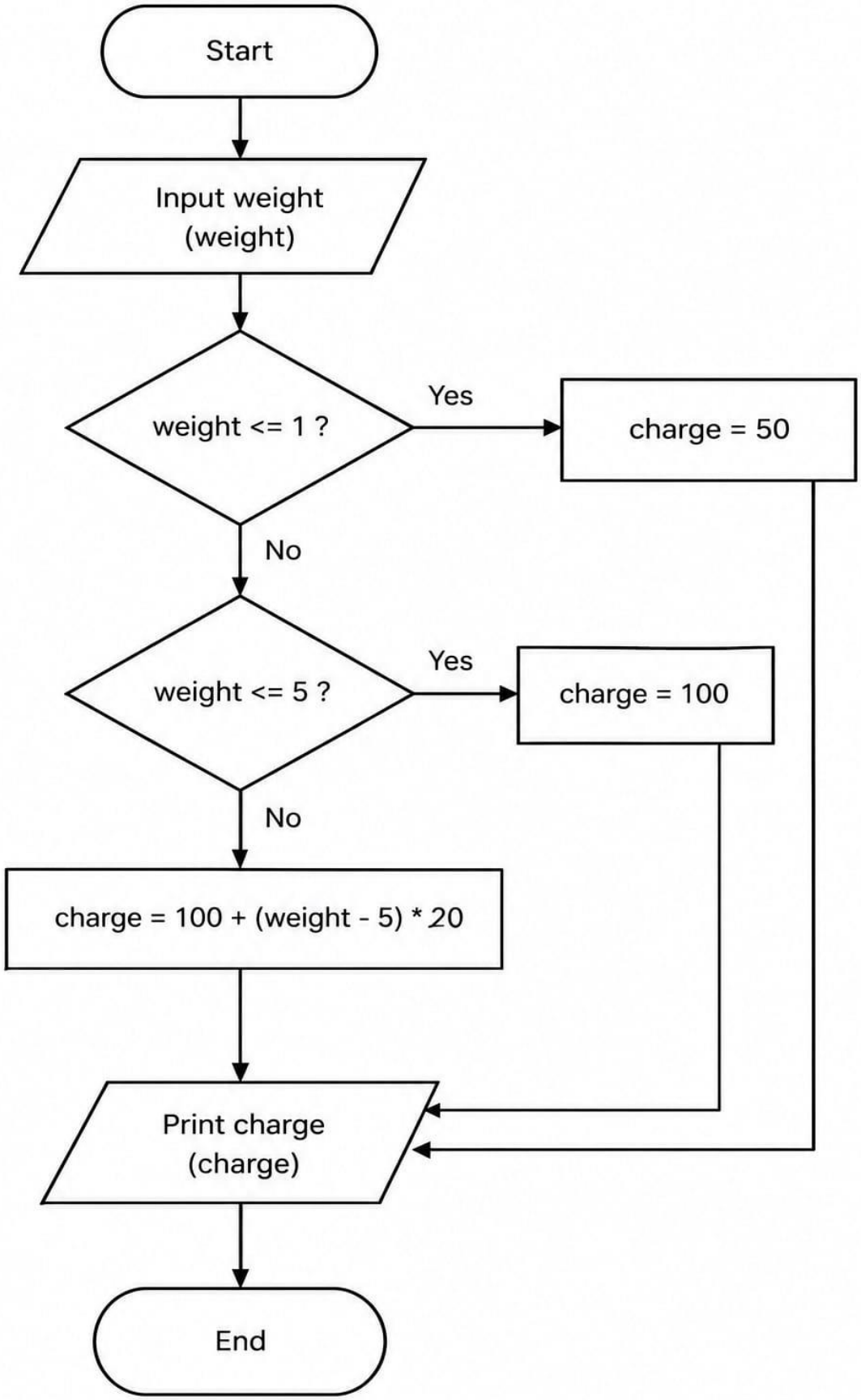
1. The parcel weight is entered by the user.
2. The weight is measured in kilograms (kg).
3. The parcel weight is a numeric value.
4. Parcels weighing up to 1 kg cost ₹50.
5. Parcels weighing above 1 kg and up to 5 kg cost ₹100.
6. Parcels weighing above 5 kg cost ₹100 plus ₹20 for each additional kilogram.
7. The parcel weight is assumed to be positive.

4. Algorithm

1. Start.
2. Read the parcel weight.
3. Check whether the weight is less than or equal to 1 kg.
4. If true, set the charge to ₹50.
5. Otherwise, check whether the weight is less than or equal to 5 kg.
6. If true, set the charge to ₹100.
7. Otherwise, calculate the charge using the formula:
   charge = 100 + (weight - 5) × 20
8. Display the courier charge.

9. Stop.

5.flowchart


Start
Input weight
(weight)
weight <= 1 ?
Yes
charge = 50
No
weight <= 5 ?
Yes
charge = 100
No
charge = 100 + (weight - 5) * 20
Print charge
(charge)
End

6. Pseudocode

START

INPUT weight

IF weight <= 1 THEN

charge = 50

ELSE IF weight <= 5 THEN

charge = 100

ELSE

charge = 100 + (weight - 5) * 20

END IF

PRINT "Courier Charge:", charge

STOP

7.Python Source Code

```
weight = float(input())

if weight <= 1:
    charge = 50
elif weight <= 5:
    charge = 100
else:
    charge = 100 + (weight - 5) * 20

print("Courier Charge:", charge)
```

8.Test Cases

Test case 1

0.6

Courier Charge: 50

Test case 2

5.0

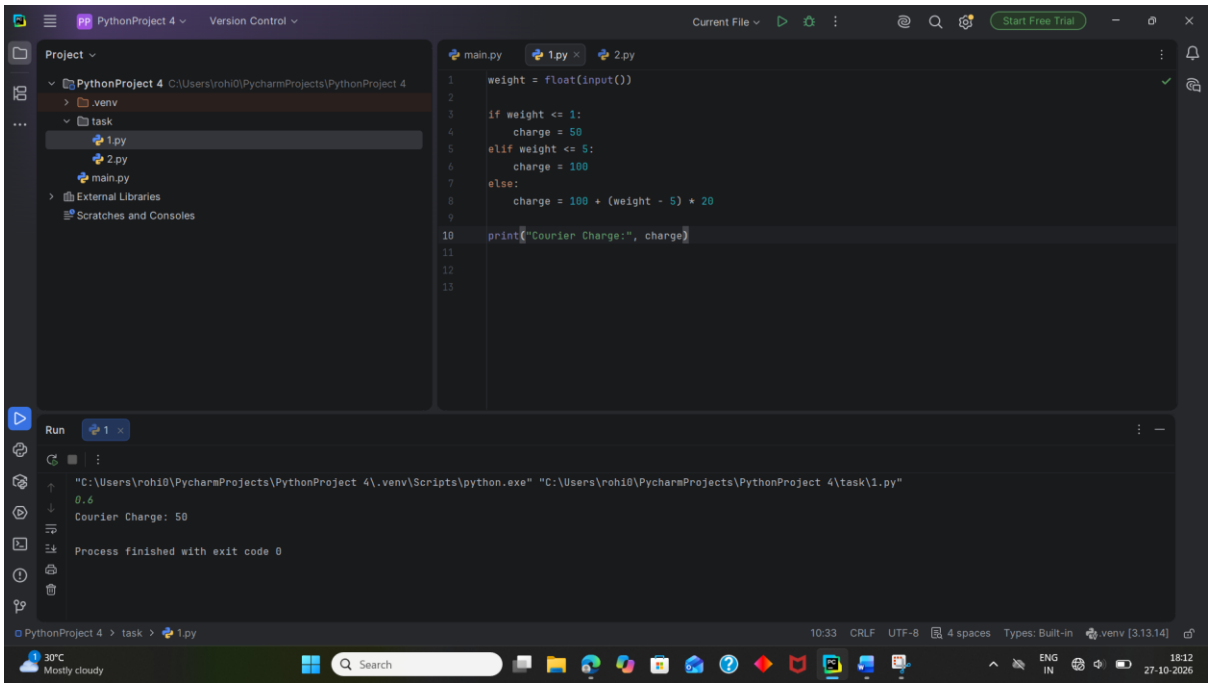
Courier Charge: 100

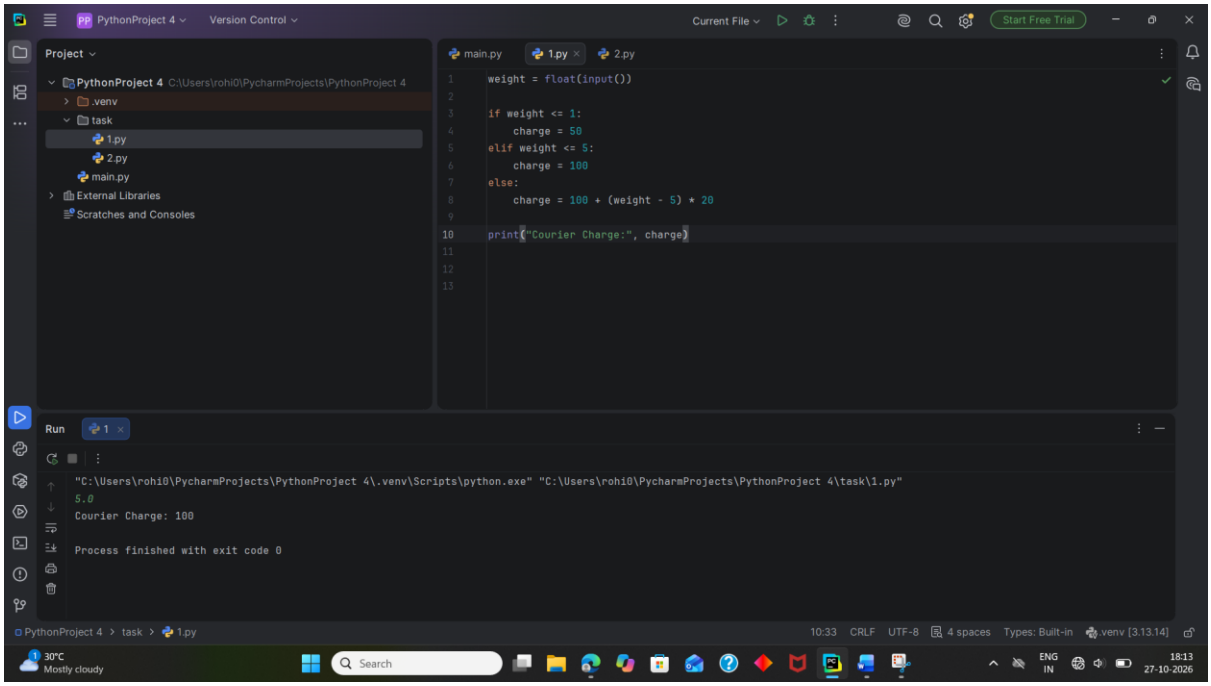
Test case 3

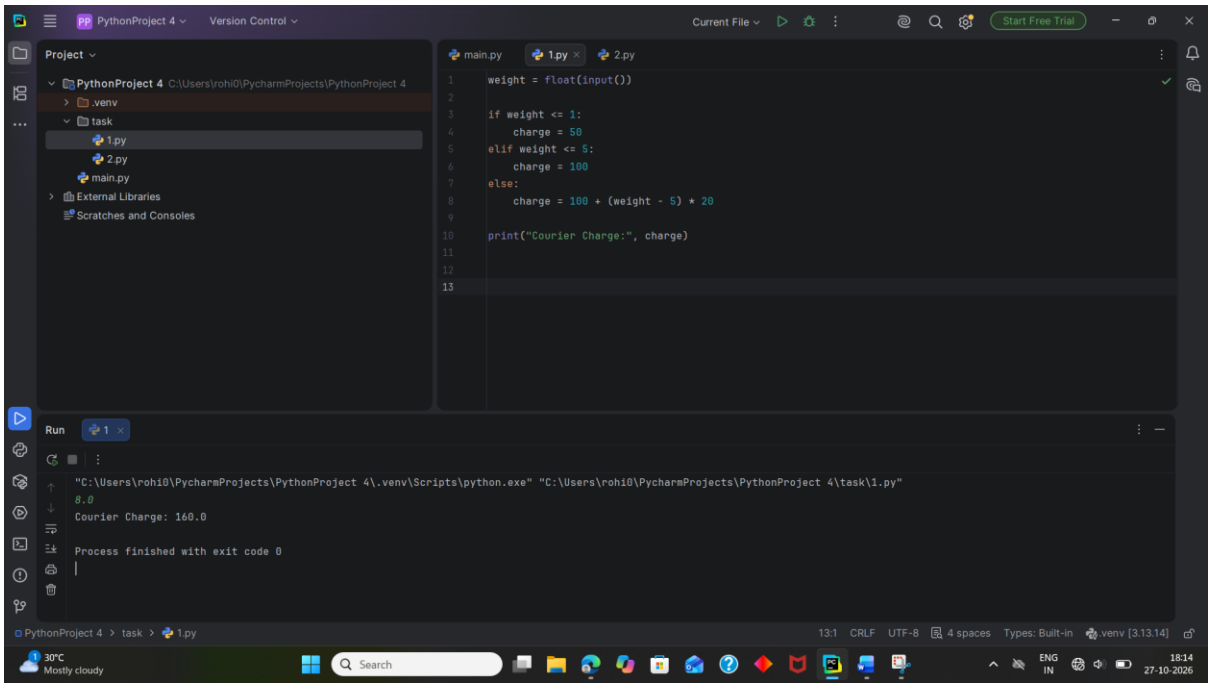
8.0

Courier Charge: 160.0

9.output







10. Conclusion

The Automated Courier Charge Calculation System calculates parcel delivery charges based on weight. It uses conditional statements and arithmetic operations to determine the correct charge for different weight ranges.

Case Study 7: Student Email and ID Validation System

1. Problem Statement

Develop a Python program to validate a student's email address and student ID. The program should accept an email address and student ID from the user. If the email contains both @ and . and the student ID contains only letters and numbers, display **"Valid Details"**. Otherwise, display **"Invalid Details"**.

2. Objective

1. To validate student email addresses and student IDs.
2. To understand the use of conditional statements in Python.
3. To check whether a string contains specific characters.
4. To use the isalnum() method to validate student IDs.
5. To display the appropriate validation result.

3. Assumptions

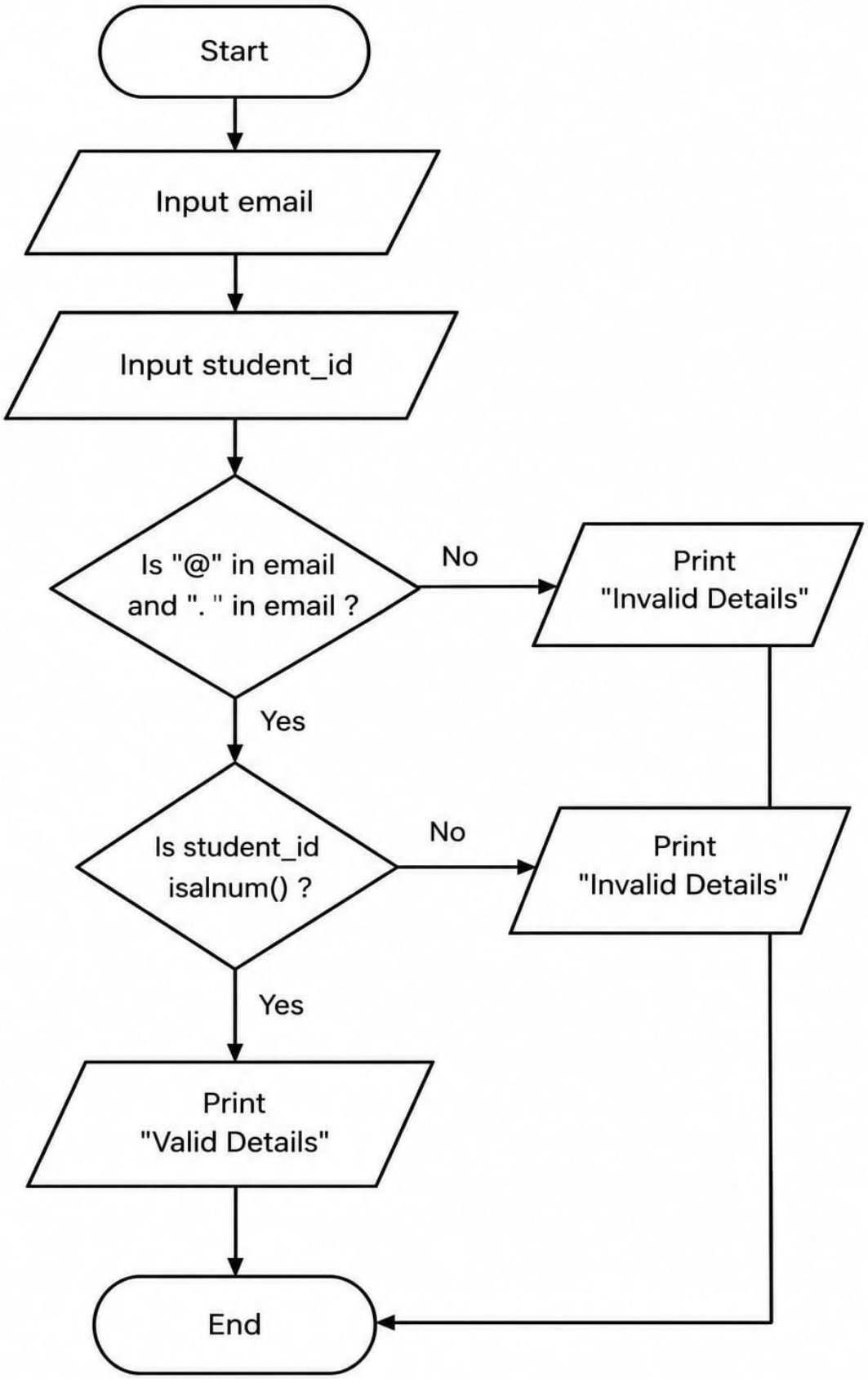
1. The user enters an email address and student ID.
2. The email address must contain @ and ..
3. The student ID must contain only letters and numbers.
4. The student ID must not contain spaces or special characters.
5. Both inputs are entered as strings.
6. The program performs basic validation and does not guarantee that the email address actually exists.

4. Algorithm

1. Start.
2. Read the email address.
3. Read the student ID.
4. Check whether the email contains @.
5. Check whether the email contains ..
6. Check whether the student ID contains only letters and numbers using isalnum().
7. If all three conditions are true, display **"Valid Details"**.
8. Otherwise, display **"Invalid Details"**.

9. Stop.

5.Flowchart


Start
Input email
Input student_id
Is "@" in email and ". " in email ?
No
Print "Invalid Details"
Yes
Is student_id isalnum() ?
No
Print "Invalid Details"
Yes
Print "Valid Details"
End

6. Pseudocode

START

INPUT email

INPUT student_id

IF email contains "@" AND

email contains "." AND

student_id is alphanumeric THEN

PRINT "Valid Details"

ELSE

PRINT "Invalid Details"

END IF

STOP

7.Python Source Code

```
email = input()
student_id = input()

if "@" in email and "." in email and student_id.isalnum():
    print("Valid Details")
else:
    print("Invalid Details")
```

8.Test Cases

Test Case 1

student@gmail.com

csm3

Valid Details

Test Case 2

studentgmail.com

cse2

Invalid Details

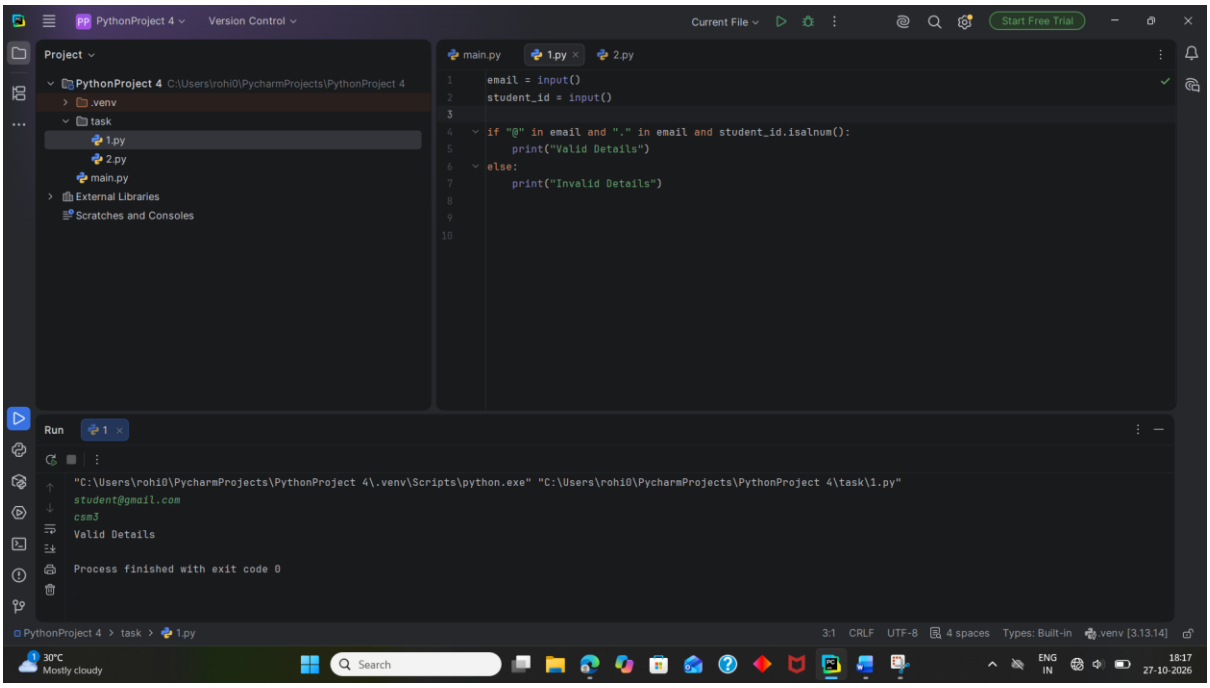
Test case 3

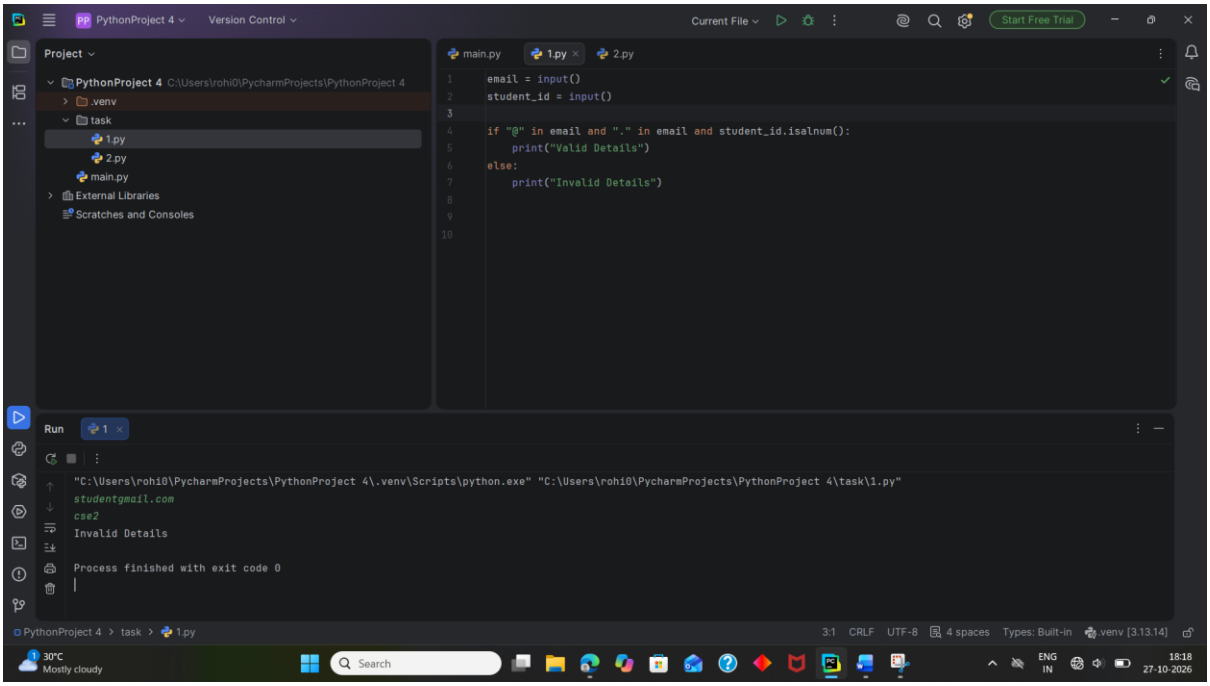
student@gmail.com

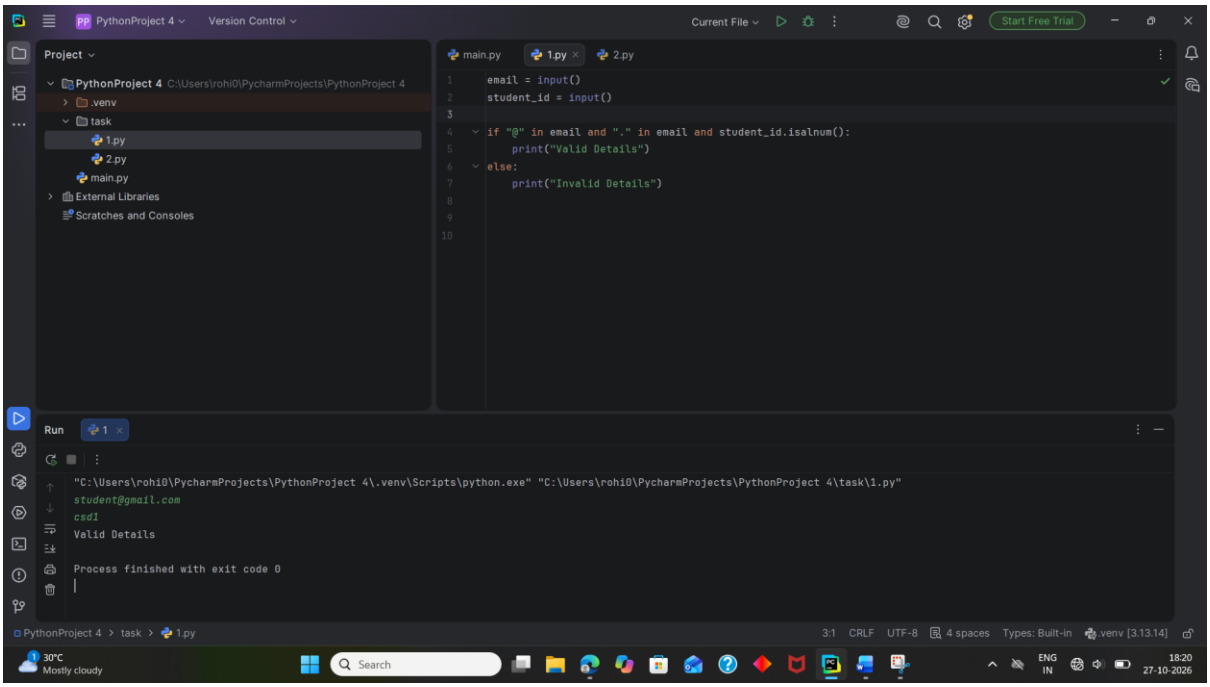
csd1

Valid Details

9. Output







10. Conclusion

The Student Email and ID Validation System checks the basic validity of email addresses and student IDs using Python conditional statements, string membership operators, and the isalnum() method. It displays the appropriate result based on the entered details.

Case Study 8: Calendar Generation Using Nested Loops

1. Problem Statement

Develop a Python program to generate a simple calendar by accepting the number of days from the user. The program should display the weekdays from Monday to Sunday and print the day numbers in sequence. After every seven days, the program should move to the next line.

2. Objective

1. To generate a simple calendar using Python.
2. To understand the use of for loops.
3. To use the modulus operator (%) to identify every seventh day.
4. To display day numbers in a structured format.
5. To understand the use of if statements and the print() function.

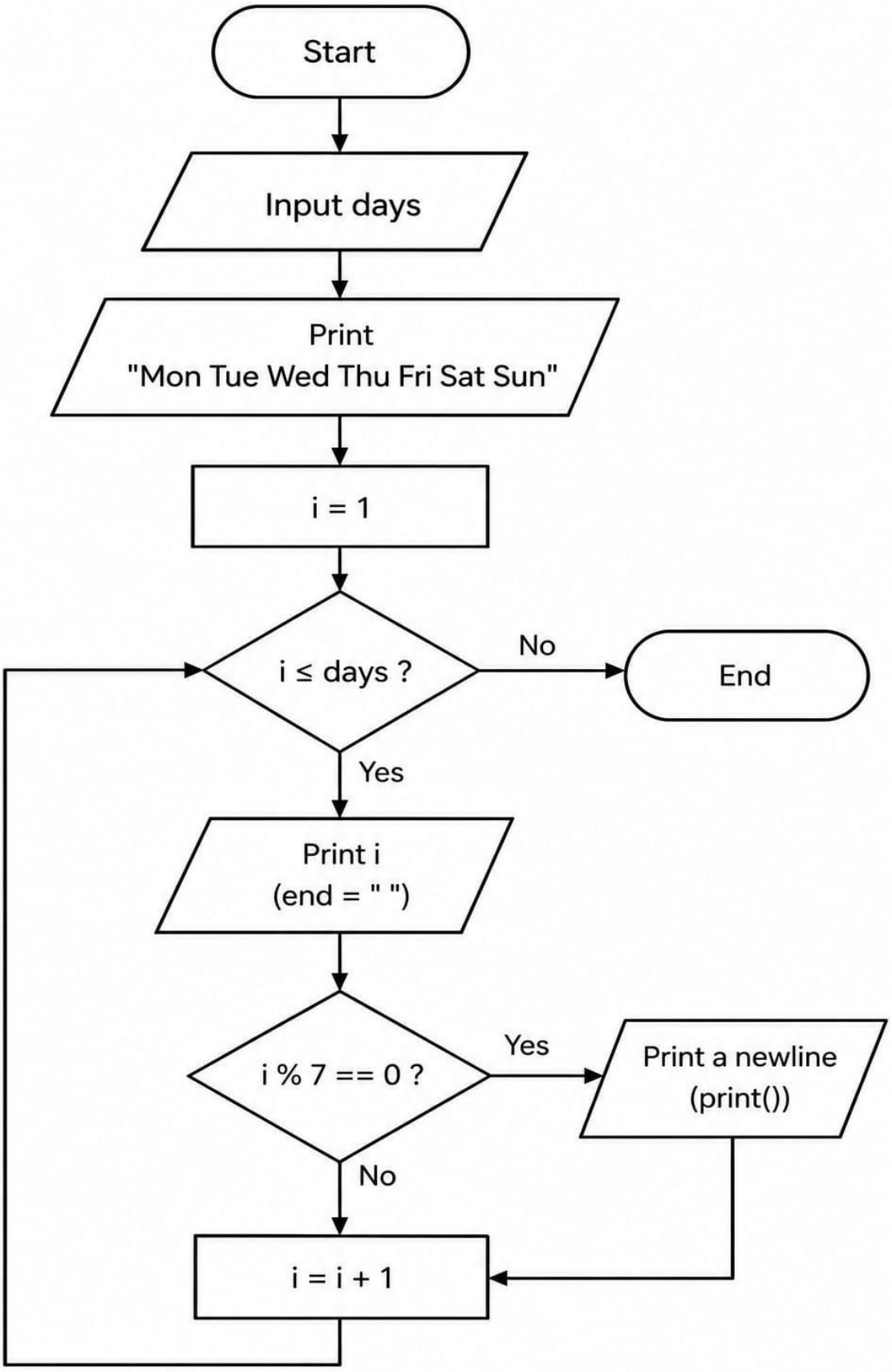
3. Assumptions

1. The user enters the total number of days as an integer.
2. The number of days is a positive integer.
3. The calendar starts with Monday.
4. The weekday names are displayed from Monday to Sunday.
5. A new line is printed after every seven days.
6. The program generates a simple calendar and does not account for the actual starting weekday of a month.

4. Algorithm

1. Start.
2. Read the number of days.
3. Display the weekday names from Monday to Sunday.
4. Initialize the loop variable i from 1 to the number of days.
5. Print the current day number on the same line.
6. Check whether i % 7 == 0.
7. If true, move to the next line.
8. Repeat until all days are printed.
9. Stop.

5.Flowchart


Start
Input days
Print
"Mon Tue Wed Thu Fri Sat Sun"
i = 1
i ≤ days ?
No
End
Yes
Print i
(end = " ")
i % 7 == 0 ?
Yes
Print a newline
(print())
No
i = i + 1

6. Pseudocode

START

INPUT days

PRINT "Mon Tue Wed Thu Fri Sat Sun"

FOR i = 1 TO days

PRINT i WITHOUT NEWLINE

IF i MOD 7 = 0 THEN

PRINT NEWLINE

END IF

END FOR

STOP

7.Python Source Code

```
days = int(input())

print("Mon Tue Wed Thu Fri Sat Sun")

for i in range(1, days + 1):
    print(i, end=" ")
    if i % 7 == 0:
        print()
```

8.Test cases

Test case 1

5

Mon Tue Wed Thu Fri Sat Sun

1 2 3 4 5

Test case 2

15

Mon Tue Wed Thu Fri Sat Sun

1 2 3 4 5 6 7

8 9 10 11 12 13 14

15

Test case 3

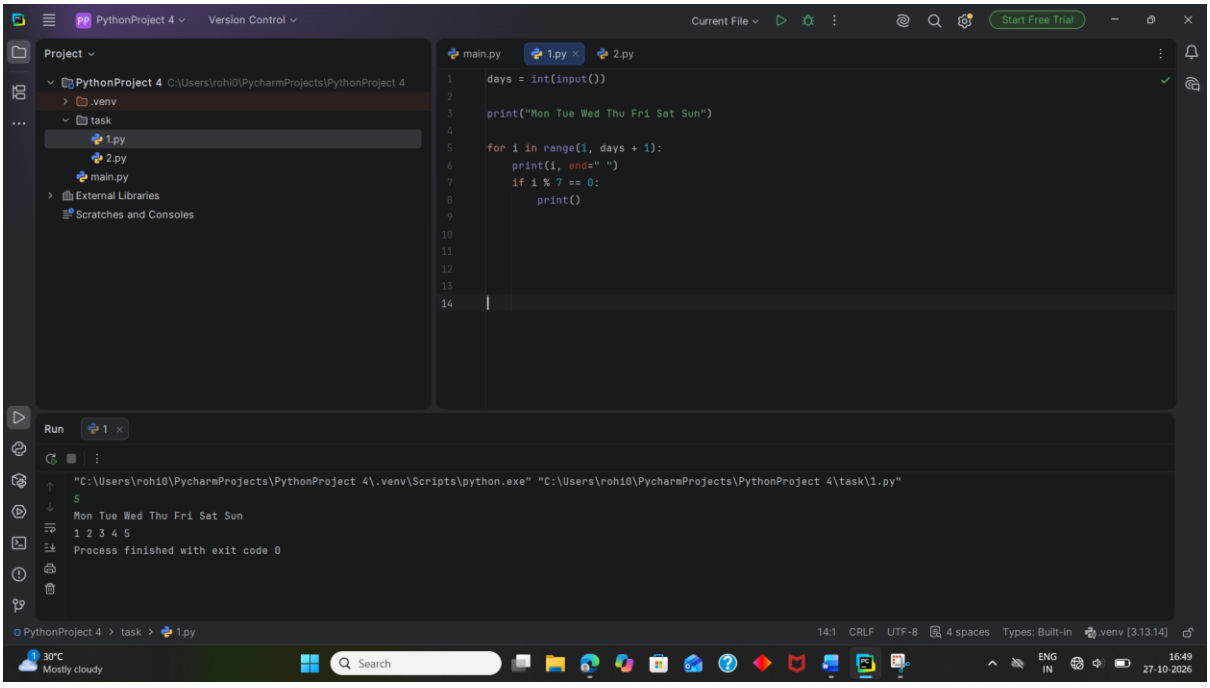
9

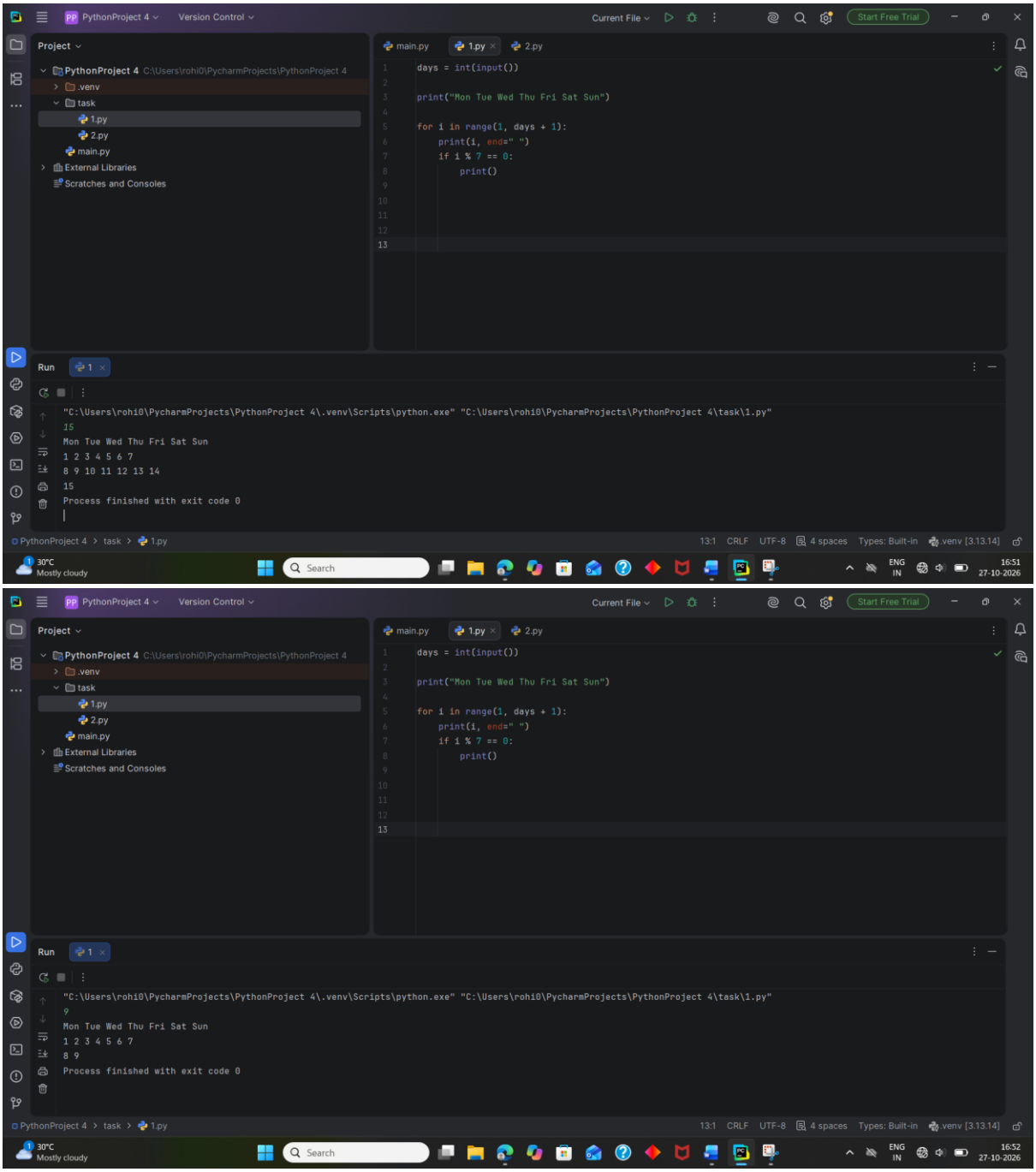
Mon Tue Wed Thu Fri Sat Sun

1 2 3 4 5 6 7

8 9

9.Output





10. Conclusion

The Calendar Generation System displays weekday names and day numbers using Python loops and conditional statements. The modulus operator identifies every seventh day, allowing the program to organize the day numbers into weekly rows.

Case Study 9: University Scholarship Allocation System

1. Problem Statement

Develop a Python program to determine a student's scholarship eligibility based on their marks and family income. If the student scores 90 or above and their family income is less than or equal to ₹2,00,000, they receive a 100% scholarship. If the student scores 80 or above and their family income is less than or equal to ₹3,00,000, they receive a 50% scholarship. Otherwise, the student receives no scholarship. Finally, display the student's name, marks, family income, and scholarship status.

2. Objective

1. To determine scholarship eligibility based on marks and family income.
2. To understand the use of if, elif, and else statements.
3. To apply logical operators such as and in Python.
4. To display student details and scholarship results.
5. To implement a simple scholarship allocation system.

3. Assumptions

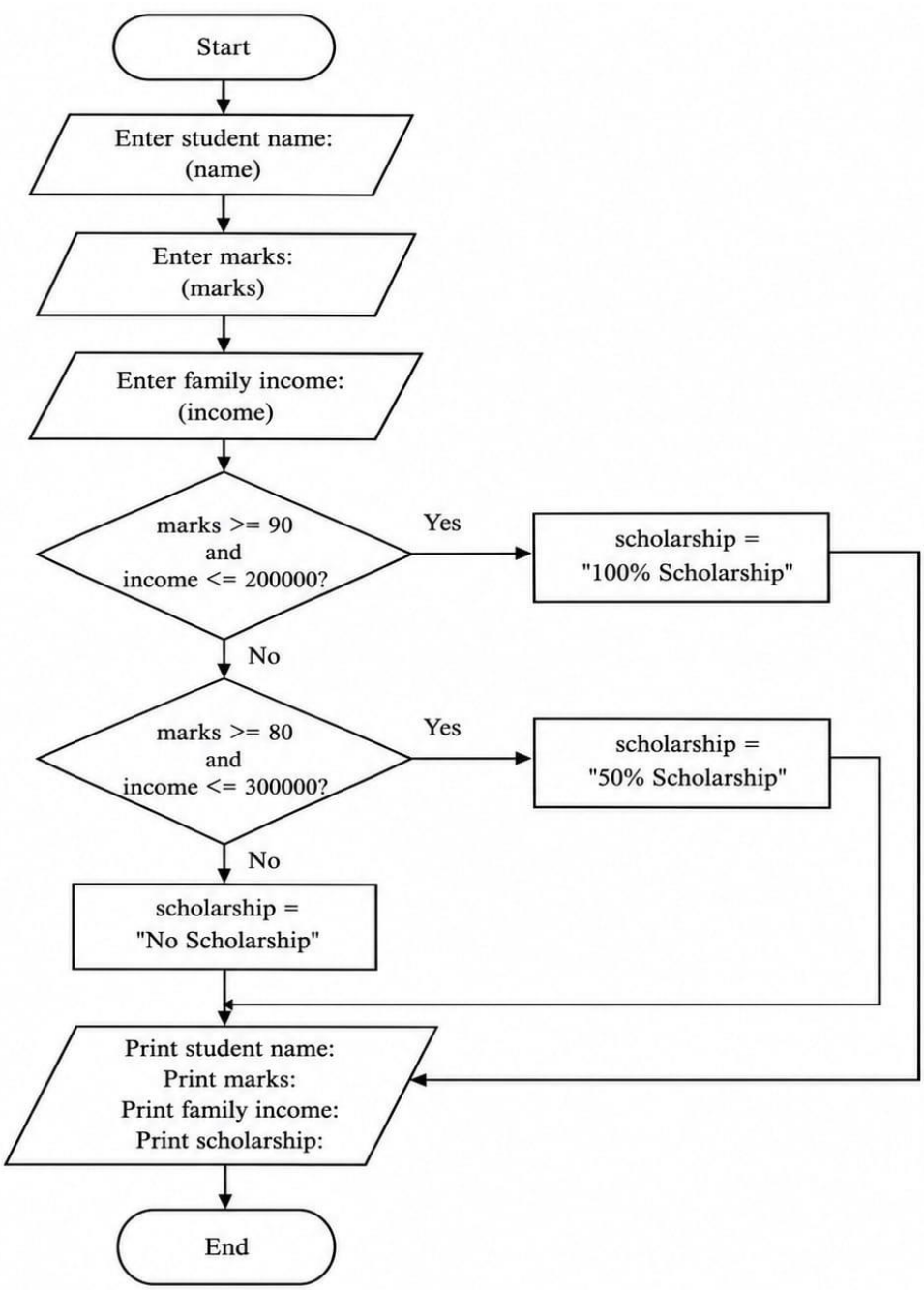
1. The student enters their name, marks, and family income.
2. Marks are entered as an integer between 0 and 100.
3. Family income is entered as a non-negative integer in rupees.
4. Students scoring 90 or above with family income up to ₹2,00,000 receive a 100% scholarship.
5. Students scoring 80 or above with family income up to ₹3,00,000 receive a 50% scholarship, provided they do not qualify for the first category.
6. Students who do not satisfy either condition receive no scholarship.

4. Algorithm

1. Start.
2. Input the student's name.
3. Input the student's marks.
4. Input the family's annual income.
5. Check whether marks are greater than or equal to 90 and income is less than or equal to ₹2,00,000.
6. If true, assign **100% Scholarship**.

7. Otherwise, check whether marks are greater than or equal to 80 and income is less than or equal to ₹3,00,000.
8. If true, assign **50% Scholarship**.
9. Otherwise, assign **No Scholarship**.
10. Display the student's name, marks, family income, and scholarship status.
11. Stop.

5.Flowchart



6. Pseudocode

```
START

INPUT name
INPUT marks
INPUT income

IF marks >= 90 AND income <= 200000 THEN
   scholarship = "100% Scholarship"

ELSE IF marks >= 80 AND income <= 300000 THEN
   scholarship = "50% Scholarship"

ELSE
   scholarship = "No Scholarship"

END IF

PRINT "Student Name:", name
PRINT "Marks:", marks
PRINT "Family Income:", income
PRINT "Scholarship:", scholarship

STOP
```

7. Python Source Code

```
name = input("Enter student name: ")
marks = int(input("Enter marks: "))
income = int(input("Enter family income: "))
```

```
if marks >= 90 and income <= 200000:
    scholarship = "100% Scholarship"
elif marks >= 80 and income <= 300000:
    scholarship = "50% Scholarship"
else:
    scholarship = "No Scholarship"

print("Student Name:", name)
print("Marks:", marks)
print("Family Income:", income)
print("Scholarship:", scholarship)
```

8.Test cases

Test case 1

Enter student name: Ramesh

Enter marks: 95

Enter family income: 150000

Student Name: Ramesh

Marks: 95

Family Income: 150000

Scholarship: 100% Scholarship

Test case 2

Enter student name: Teja sree

Enter marks: 85

Enter family income: 250000

Student Name: Teja sree

Marks: 85

Family Income: 250000

Scholarship: 50% Scholarship

Test case 3

Enter student name: Hanifa

Enter marks: 75

Enter family income: 180000

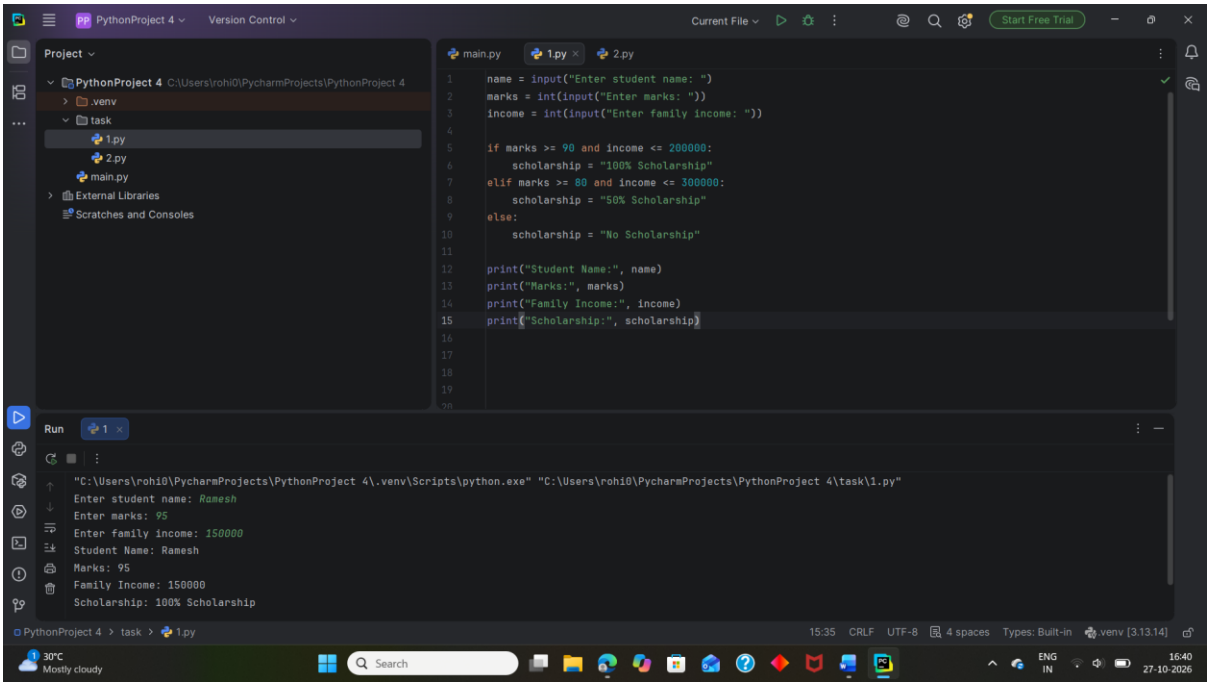
Student Name: Hanifa

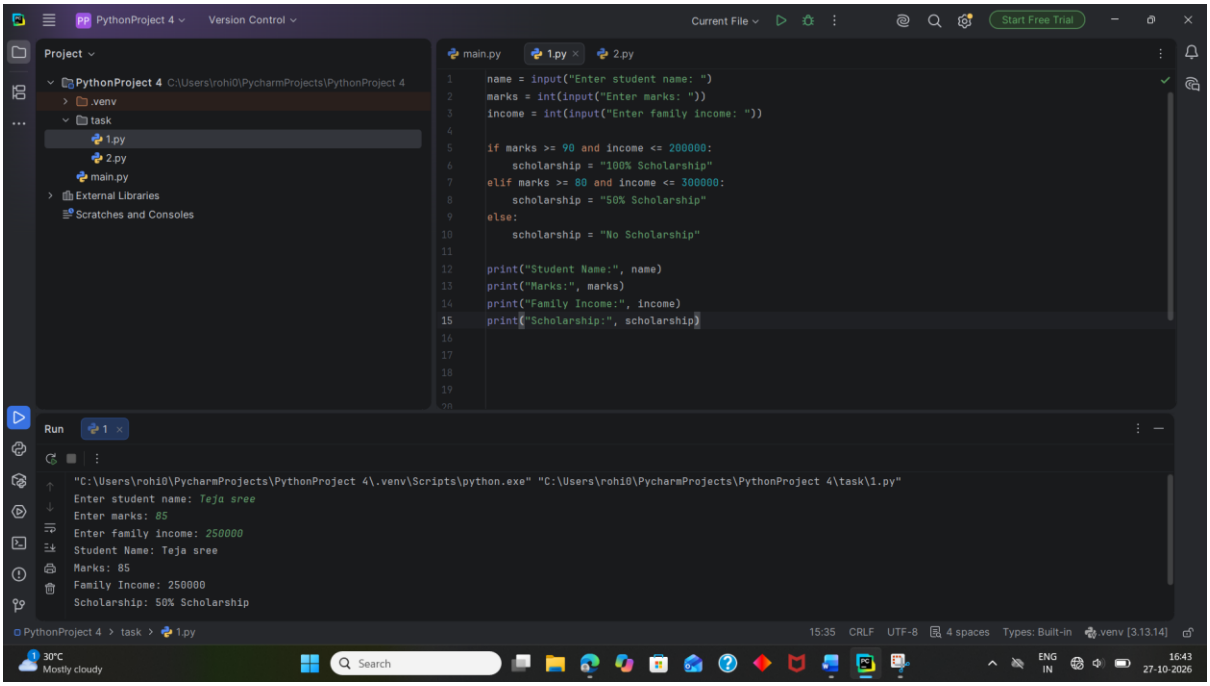
Marks: 75

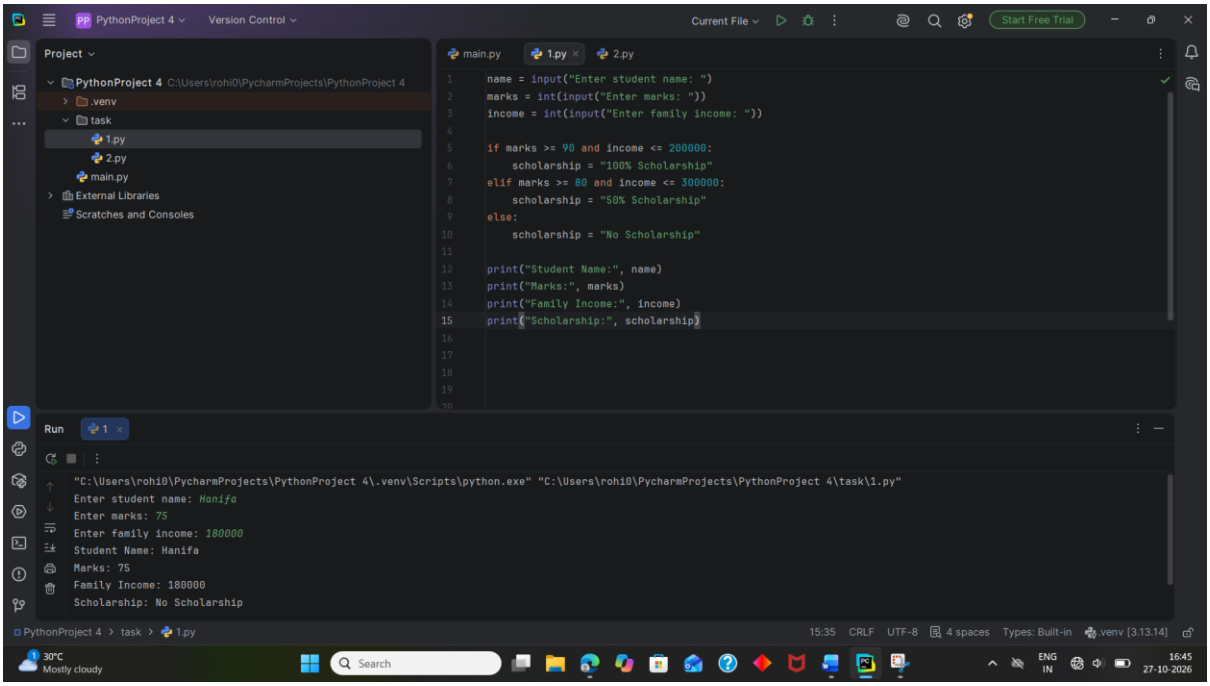
Family Income: 180000

Scholarship: No Scholarship

9.Sample Output







10. Conclusion

The University Scholarship Allocation System determines scholarship eligibility based on students' marks and family income. It uses conditional statements and the logical and operator to assign the appropriate scholarship category. This program demonstrates the practical application of Python decision-making statements.

Case Study 10: Campus Transport Allocation System

1. Problem Statement

Develop a Python program to allocate campus transportation to students based on the distance between their residence and the campus. If the distance is greater than 20 km, allocate a Long-Distance Bus. If the distance is greater than 10 km but less than or equal to 20 km, allocate a Medium-Distance Bus. Otherwise, allocate a Short-Distance Bus. Finally, display the student's name, distance, and allocated transport.

2. Objective

1. To allocate campus transportation based on distance.
2. To understand the use of if, elif, and else statements.
3. To apply conditional statements to a real-world problem.
4. To accept and process student details using Python.
5. To display the allocated transportation category.

3. Assumptions

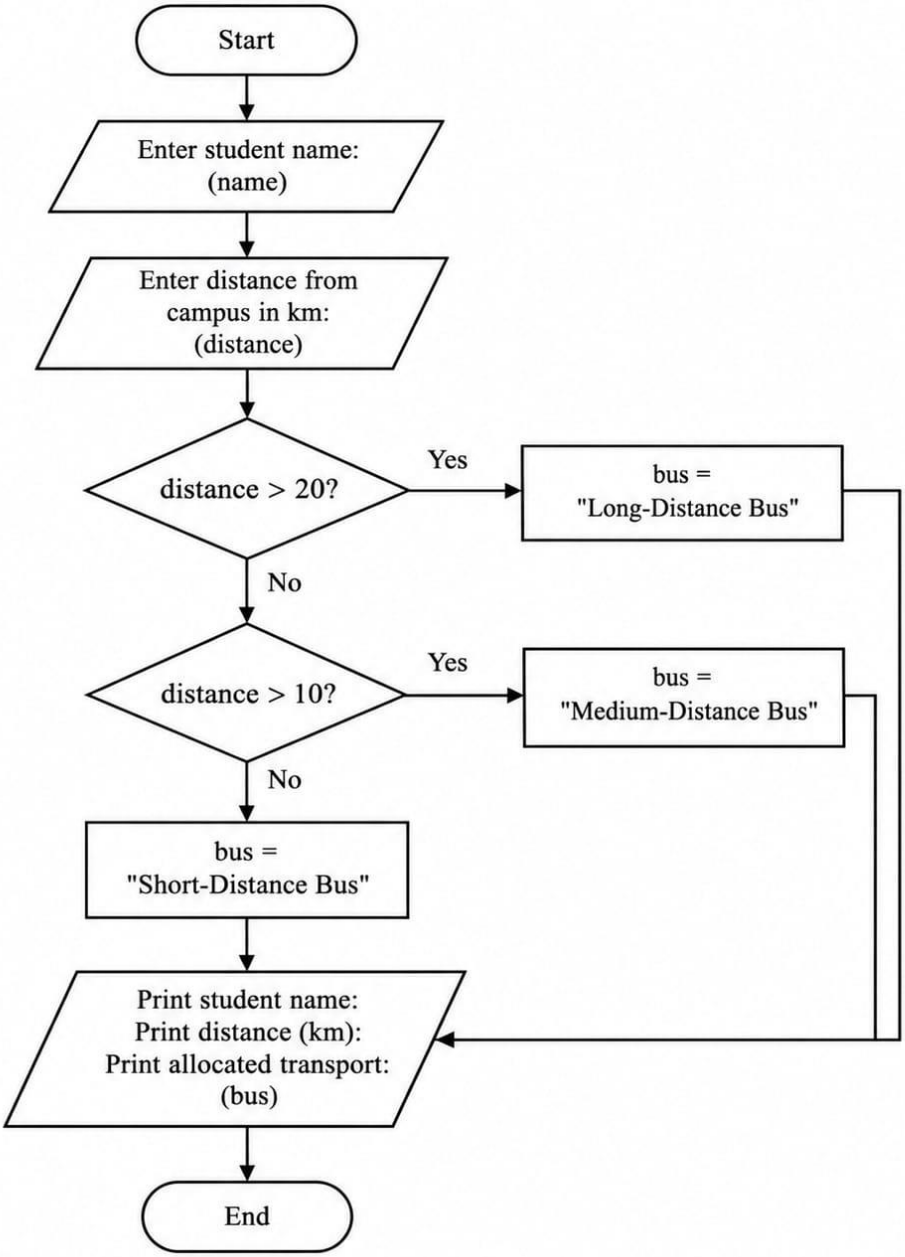
1. The student enters their name.
2. The distance from the campus is entered in kilometres.
3. The distance is a non-negative numeric value.
4. Students travelling more than 20 km receive a Long-Distance Bus.
5. Students travelling more than 10 km and up to 20 km receive a Medium-Distance Bus.
6. Students travelling 10 km or less receive a Short-Distance Bus.

4. Algorithm

1. Start.
2. Input the student's name.
3. Input the distance from the campus in kilometres.
4. Check whether the distance is greater than 20 km.
5. If true, allocate the Long-Distance Bus.
6. Otherwise, check whether the distance is greater than 10 km.
7. If true, allocate the Medium-Distance Bus.
8. Otherwise, allocate the Short-Distance Bus.

9. Display the student's name, distance, and allocated transport.
10. Stop.

5.Flowchart



6. Pseudocode

START

INPUT name

INPUT distance

```
IF distance > 20 THEN
    bus = "Long-Distance Bus"

ELSE IF distance > 10 THEN
    bus = "Medium-Distance Bus"

ELSE
    bus = "Short-Distance Bus"

END IF

PRINT "Student Name:", name
PRINT "Distance:", distance, "km"
PRINT "Allocated Transport:", bus

STOP
```

7.Python Source Code

```
name = input("Enter student name: ")
distance = float(input("Enter distance from campus in km: "))

if distance > 20:
    bus = "Long-Distance Bus"
elif distance > 10:
    bus = "Medium-Distance Bus"
else:
    bus = "Short-Distance Bus"

print("Student Name:", name)
print("Distance:", distance, "km")
print("Allocated Transport:", bus)
```

8.Test cases

Test case 1

```
Enter student name: Rohitha
```

Enter distance from campus in km: 30

Student Name: Rohitha

Distance: 30.0 km

Allocated Transport: Long-Distance Bus

Test case 2

Enter student name: Prasanna

Enter distance from campus in km: 10

Student Name: Prasanna

Distance: 10.0 km

Allocated Transport: Short-Distance Bus

Test case 3

Enter student name: rajesh

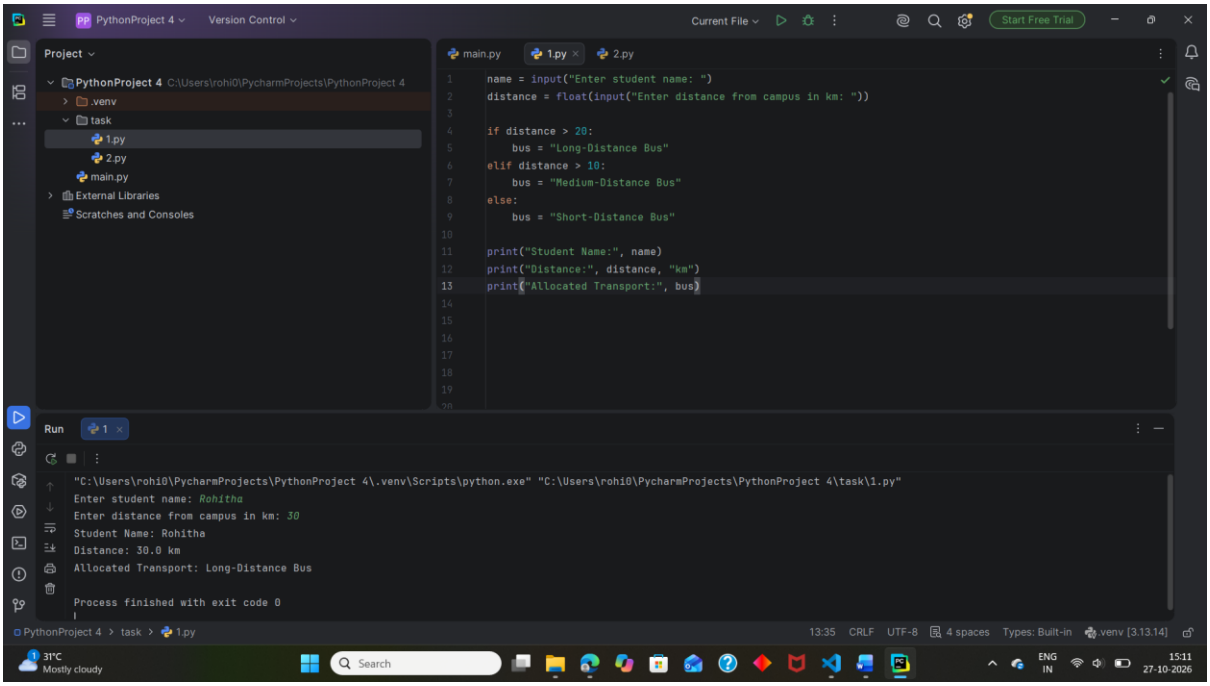
Enter distance from campus in km: 40

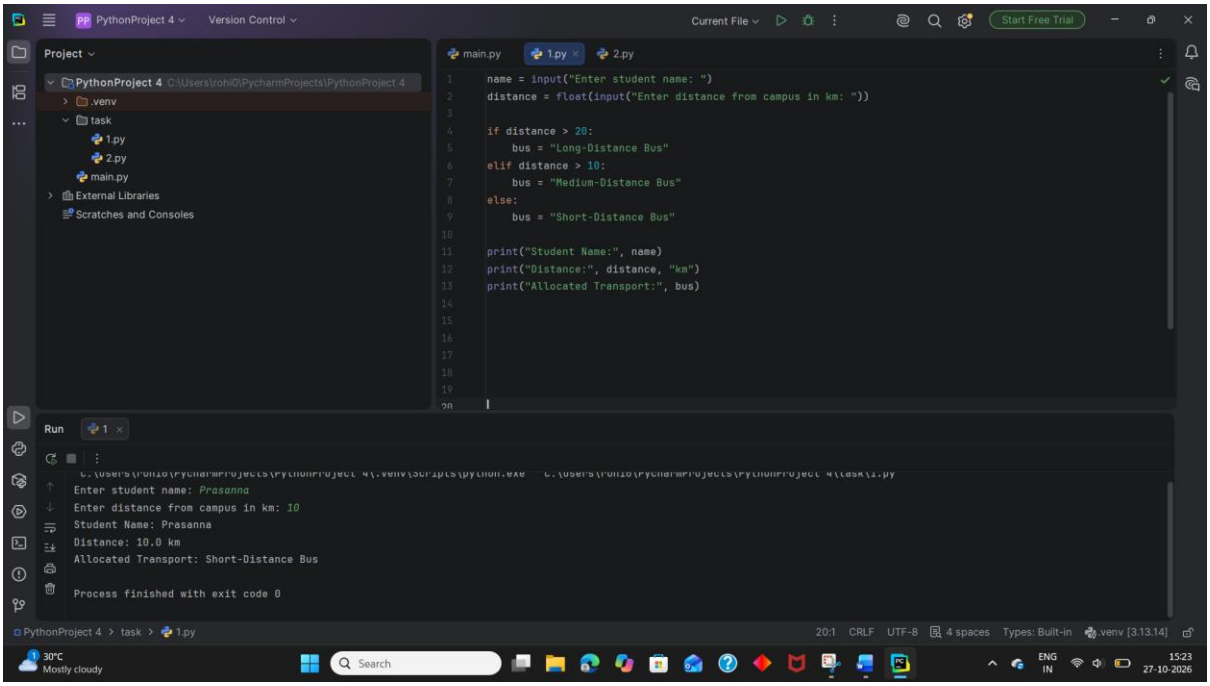
Student Name: rajesh

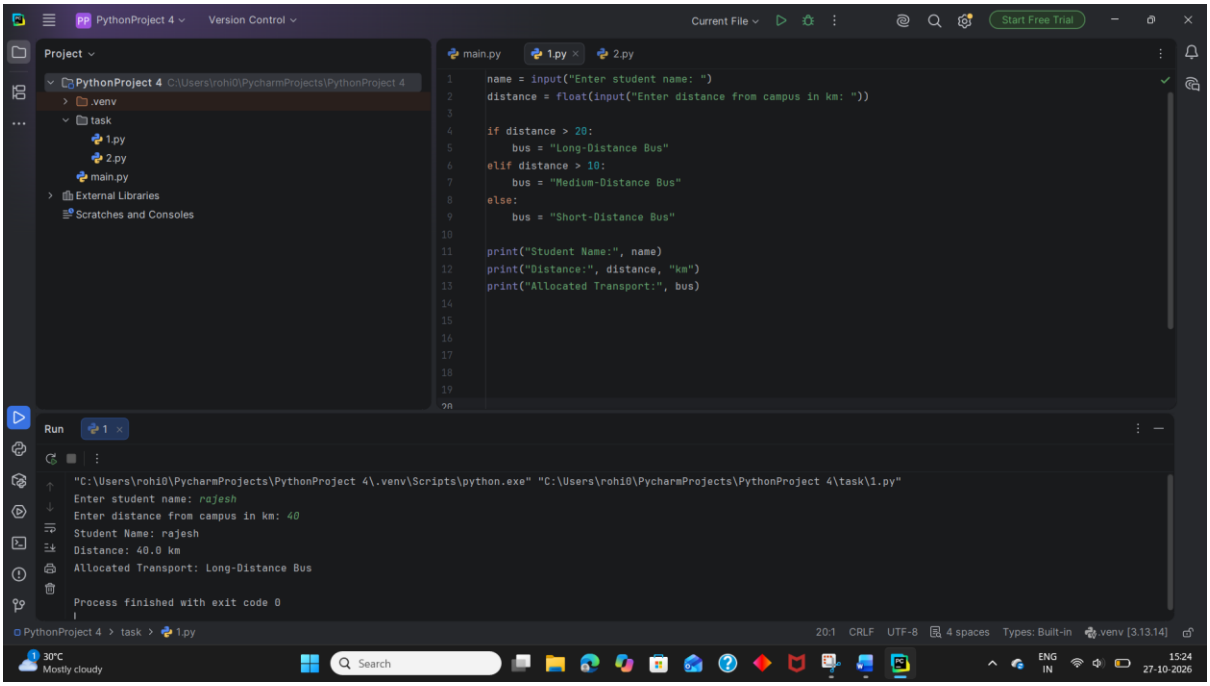
Distance: 40.0 km

Allocated Transport: Long-Distance Bus

9.Sample Output







10. Conclusion

The Campus Transport Allocation System assigns a suitable bus category based on the student's distance from campus. It uses if, elif, and else statements to evaluate distance ranges and display the appropriate transportation category.